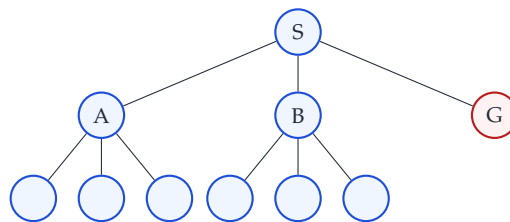

Artificial Intelligence: Search Methods for Problem Solving

Week 2 — State Space Search

Detailed study notes built on the lectures of Prof. Deepak Khemani (IIT Madras)

IITM BS in Programming & Data Science



How to use these notes: each module follows the order of the slides. Yellow boxes give intuition, blue boxes are definitions, purple boxes are formulas and derivations, black boxes are algorithms (as on the slides), green boxes are side notes, red boxes flag exam traps and slide errata. The practice section at the end (numericals, algorithm traces, subjective questions, MCQs) covers every module.

Contents

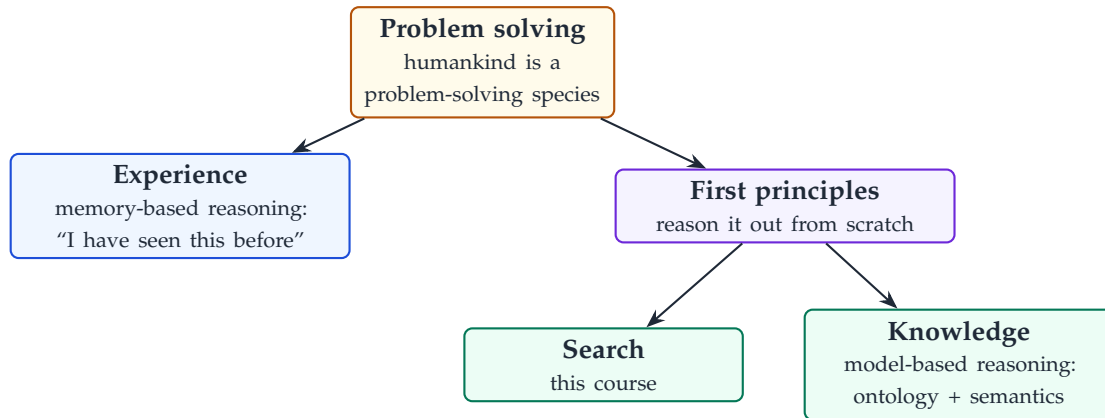
1	Recap and General-Purpose Problem Solving	3
1.1	Where we left off	3
1.2	The map colouring problem again	3
1.3	Three ways to solve map colouring	4
1.4	General-purpose methods	4
1.5	State space search is graph search	5
2	Sample Problems and Their State Spaces	7
2.1	The water jug problem	7
2.2	The Eight-puzzle	9
2.3	Man, Goat, Lion, Cabbage (MGLC)	10
2.4	The N-queens problem	12
2.5	A map colouring problem and its solutions	13
2.6	The Traveling Salesman Problem	13
2.7	A maze: path finding as graph search	14
3	General Search Algorithms	16
3.1	Domain-independent problem solving	16
3.2	The state space and its implicit nature	16
3.3	Moves: MoveGen	16
3.4	A search node is like a junction in a maze	17
3.5	OPEN and CLOSED	17
3.6	Search = treasure hunt: generate and test	17
3.7	A tiny search problem	18
3.8	The search space is a tree	18
3.9	SimpleSearch1	19
3.10	SimpleSearch2: CLOSED, a repository of seen nodes	19
3.11	Even when it succeeds, search does not return the path	21
4	Solutions as Paths: NodePairs, Determinism and Notation	22
4.1	Planning problems vs configuration problems	22
4.2	NodePairs: keeping track of parents	22
4.3	Reconstructing the path	23
4.4	A search program generates and explores a search tree	23
4.5	Deterministic search algorithms	24
4.6	List notation	24
4.7	Tuple notation	25

5	Depth First and Breadth First Search	26
5.1	Depth First Search: OPEN is a stack	26
5.2	Ancillary functions	26
5.3	Breadth First Search: OPEN is a queue	27
5.4	Stack vs queue	28
5.5	How the two searches look on a large state space	28
5.6	A tiny state space: three ways to handle duplicates	29
5.7	Running both on the tiny problem of Module 3	31
6	Analysis of DFS and BFS	32
6.1	Behaviour: the order of expansion	32
6.2	Running time depends on where the goal is	32
6.3	Time complexity	32
6.4	Space complexity: the size of OPEN	34
6.5	Quality of solution and completeness	35
6.6	Depth First vs Breadth First: the summary table	35
7	The Best of Both: Depth-First Iterative Deepening	37
7.1	Depth Bounded DFS (DBDFS)	37
7.2	DBDFS-2: returning a count of nodes visited	38
7.3	Depth First Iterative Deepening (DFID)	38
7.4	Does DFID return the shortest path? Careful with CLOSED	39
7.5	“Is there a catch?” The cost of extra work	41
7.6	The final comparison	42
7.7	CombEx: combinatorial explosion	42
7.8	Blind search: same behaviour irrespective of the goal	42
	Practice: Test Yourself	44
	Part A — Numerical problems and traces	44
	Part B — Subjective questions	48
	Part C — MCQs	51

Module 1 Recap and General-Purpose Problem Solving

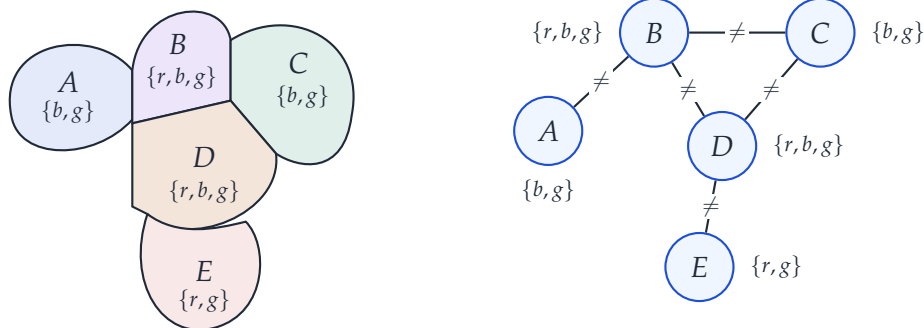
1.1 Where we left off

Week 1 ended with two ways to solve a problem. The recap slide repeats the picture:



This week builds the machinery of the “search” box: how to describe *any* problem so that one general algorithm can solve it, and the first two such algorithms, depth-first and breadth-first search, plus their hybrid, iterative deepening.

1.2 The map colouring problem again



Colour each region with an allowed colour so that no two adjacent regions share a colour. The constraint graph has one node per region and a “ $\neq$ ” edge per border: $A-B$, $B-C$, $B-D$, $C-D$, $D-E$. (It has $2 \cdot 3 \cdot 2 \cdot 3 \cdot 2 = 72$ complete assignments, of which 8 are solutions; the Week 1 notes derive this count.)

1.3 Three ways to solve map colouring

Approach	What it does	Trade-off
Brute force	Try all combinations of colours for all regions (generate every complete assignment, test each).	Simple and general; cost is the product of domain sizes, i.e. k^n for n regions and k colours.
Informed search	Choose a colour that does not conflict with already-coloured neighbours.	Uses problem knowledge to prune early; much faster, but the program is specific to this problem.
General search	Pose the problem as input to a general search algorithm.	One algorithm serves many problems; the user only supplies a problem description.

The slide notes that map colouring can be posed in two general ways:

- as a **constraint satisfaction problem (CSP)**, done at the end of the course;
- as **state space search**, where a *state* is a partially coloured map and a *move* assigns a colour to one more region.

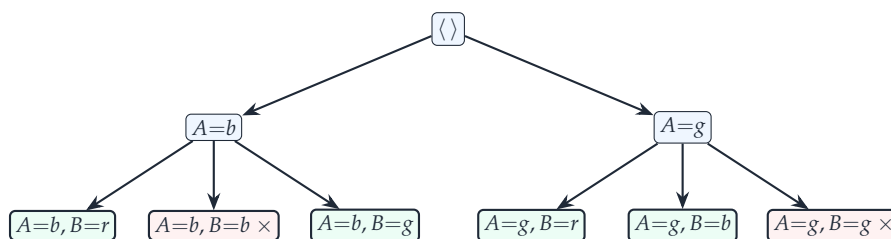
The course studies general search methods that **incorporate heuristics** to go beyond brute force.

Formula / Derivation: Map colouring as state space search: how big is the space?

Colour regions in a fixed order A, B, C, D, E . A state at depth i has the first i regions coloured. The number of states at depth i is the product of the first i domain sizes:

depth 0: 1, 1: 2, 2: $2 \cdot 3 = 6$, 3: 12, 4: 36, 5: 72.

Total nodes in the full tree = $1 + 2 + 6 + 12 + 36 + 72 = 129$. A search that rejects a partial state as soon as it breaks a constraint never expands most of these. Brute force, which only tests *complete* assignments, must look at all 72 leaves.



State space for map colouring: each move colours one more region. Red states break a constraint and need not be expanded.

1.4 General-purpose methods

Definition: General-purpose (domain-independent) method

Instead of writing a custom program for every problem, write **search algorithms into which individual problems can be plugged**.

There are two related approaches:

State space search (also called *solution space search*)

- describe the given state
- devise an operator to choose an action in each state
- navigate the state space in search of the desired (goal) state

Constraint processing

- define a set of variables, each with its own domain
- describe the constraints between subsets of variables
- search for an assignment to every variable that satisfies all constraints

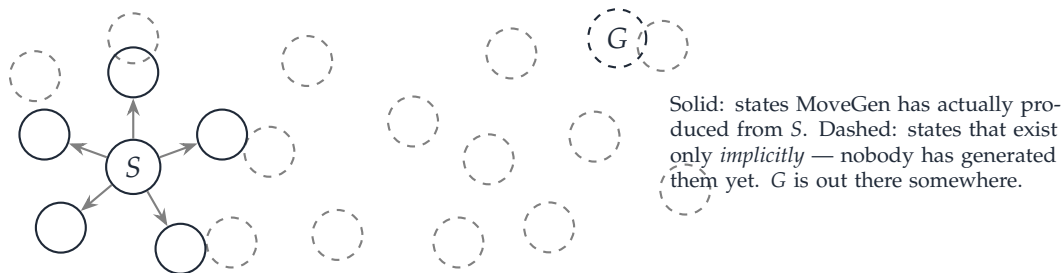
Intuition: Same problem, two lenses

State space search thinks in terms of *situations and moves*: “where am I, and where can I go from here?”. Constraint processing thinks in terms of *unknowns and restrictions*: “which values are still allowed for each unknown?”. Map colouring fits both. Route finding fits state space search naturally; Sudoku fits constraints naturally. Weeks 2–10 are mostly about the first lens; the last part of the course is about the second.

1.5 State space search is graph search

Definition: The vocabulary of state space search

- A **state** is a representation of a situation.
- A **MoveGen** function captures the moves that can be made in a given state. It returns the set of states — the **neighbours** — that result from those moves.
- The **state space** is an **implicit graph** defined by the MoveGen function: nodes are states, and there is an edge $N \rightarrow N'$ whenever $N' \in \text{MOVEGEN}(N)$.
- A **GoalTest** function checks whether a given state is a goal state.
- A **search algorithm** navigates the state space using only these two functions.



Intuition: Why “implicit” matters

For the 8-puzzle the graph has 181,440 reachable states; for chess, around 10^{44} legal positions. Nobody stores such a graph. Instead, a search algorithm is given a *rule for generating neighbours* and discovers the graph piece by piece, only in the parts it visits. This is why the course talks about MoveGen rather than an adjacency list.

Formula / Derivation: A problem = four ingredients

Formally, a state space search problem is specified by

$$\langle S, s_0 \in S, \text{MOVEGEN} : S \rightarrow 2^S, \text{GOALTEST} : S \rightarrow \{\text{true}, \text{false}\} \rangle$$

where S is the set of states (never listed explicitly), s_0 is the start state and 2^S denotes “a set of states”. A **solution** is either a goal state (for configuration problems) or a path $s_0, s_1, \dots, s_k$ with $s_{i+1} \in \text{MOVEGEN}(s_i)$ and $\text{GOALTEST}(s_k) = \text{true}$ (for planning problems). Module 4 makes this distinction precise.

Module recap

- Map colouring can be solved by brute force, by problem-specific informed choices, or by posing it to a general algorithm (as a CSP or as state space search).
- General-purpose methods separate the *algorithm* from the *problem description*.
- State space search: states, MoveGen (neighbours), GoalTest. The state space is an *implicit* graph generated on demand.

Module 2 Sample Problems and Their State Spaces

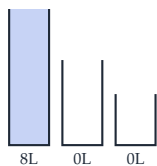
The lecture shows a gallery of problems. For each, the question to ask is always the same: **what is a state**, **what does MoveGen return**, and **what does GoalTest check**? Once those three are written, any general search algorithm can be applied.

2.1 The water jug problem

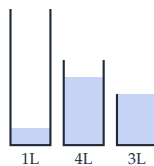
Definition: Water jug problem (8–5–3)

Three jugs with capacities 8, 5 and 3 litres. **Start**: the 8-litre jug is full, the others empty. **Goal**: measure 4 litres. No water may be thrown away and there are no markings, so the only move is to pour from one jug into another until the source is empty or the destination is full.

Given (8,0,0)

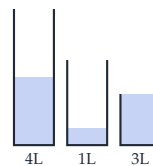


(1,4,3)

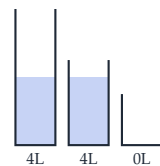


Desired: any of these

(4,1,3)



(4,4,0)



2.1.1 Representation

A state is a triple (a, b, c) : litres in the 8-, 5- and 3-litre jugs. Two invariants hold in every reachable state:

$$a + b + c = 8 \quad (\text{no water created or lost}), \quad 0 \leq a \leq 8, 0 \leq b \leq 5, 0 \leq c \leq 3.$$

Formula / Derivation: The pour operation — MoveGen for water jugs

Pouring from jug i into jug j ($i \neq j$) moves

$$t = \min(x_i, \text{cap}_j - x_j)$$

litres: either the source empties ($t = x_i$) or the destination fills ($t = \text{cap}_j - x_j$), whichever comes first. The new state has $x_i \leftarrow x_i - t$, $x_j \leftarrow x_j + t$. A pour is a real move only if $t > 0$.

With 3 jugs there are $3 \times 2 = 6$ ordered (source, destination) pairs, so every state has **at most 6** neighbours.

Algorithm: MoveGen and GoalTest for the water jug problem (answer to the slide exercise)

```

1: function MOVEGEN( $(x_1, x_2, x_3)$ )                                ▷ capacities cap = (8,5,3)
2:   neighbours  $\leftarrow []$ 
3:   for each source  $i$  and destination  $j \neq i$  do
4:      $t \leftarrow \min(x_i, \text{cap}_j - x_j)$ 
5:     if  $t > 0$  then
6:        $y \leftarrow x$ ;  $y_i \leftarrow x_i - t$ ;  $y_j \leftarrow x_j + t$ 
7:       neighbours  $\leftarrow y : \text{neighbours}$ 
8:     end if
9:   end for
10:  return neighbours

```

```

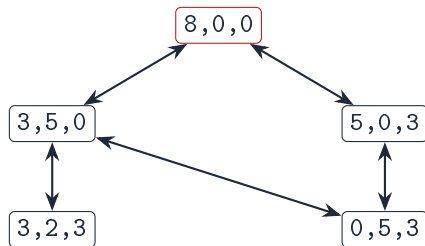
11: end function
12: function GOALTEST(( $x_1, x_2, x_3$ ))
13:   return ( $x_1 = 4$ ) or ( $x_2 = 4$ ) or ( $x_3 = 4$ )
14: end function

```

▷ “any jug has 4 litres”

2.1.2 Sample moves and the state transition graph

From $(8, 0, 0)$ only two pours are possible: $8 \rightarrow 5$ giving $(3, 5, 0)$, and $8 \rightarrow 3$ giving $(5, 0, 3)$. From these, more states appear:



$(8, 0, 0) \xrightarrow{8 \rightarrow 5} (3, 5, 0) \xrightarrow{5 \rightarrow 3} (3, 2, 3)$

$(8, 0, 0) \xrightarrow{8 \rightarrow 3} (5, 0, 3) \xrightarrow{8 \rightarrow 5} (0, 5, 3)$

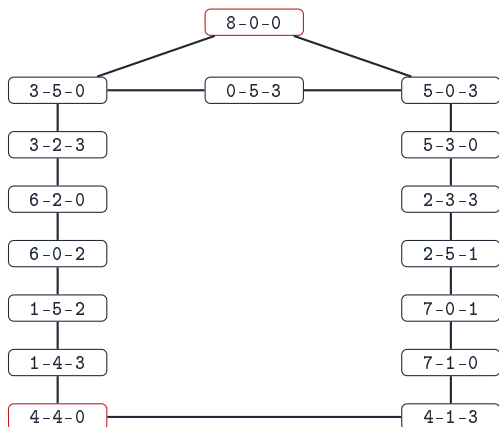
$(3, 5, 0) \xrightarrow{8 \rightarrow 3} (0, 5, 3)$

Double arrows: both directions are single pours

(e.g. $(3, 5, 0) \xrightarrow{5 \rightarrow 8} (8, 0, 0)$).

2.1.3 The solution: $(8, 0, 0) \rightarrow (4, 4, 0)$

Various paths reach $(4, 4, 0)$. The slide highlights two, which go round the two sides of the state space:



Left path (7 moves):

8-0-0, 3-5-0, 3-2-3, 6-2-0, 6-0-2, 1-5-2, 1-4-3, 4-4-0.

Right path (8 moves):

8-0-0, 5-0-3, 5-3-0, 2-3-3, 2-5-1, 7-0-1, 7-1-0, 4-1-3, 4-4-0.

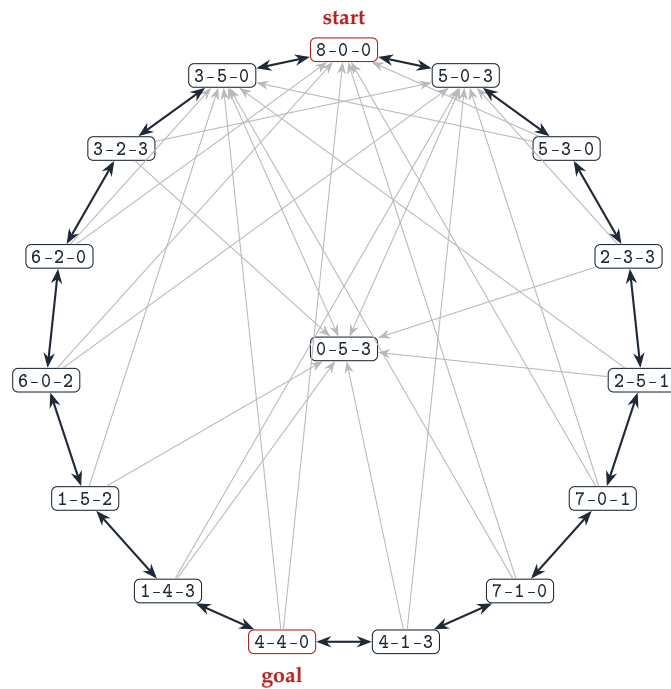
Note $(1, 4, 3)$ already has 4 litres after 6 moves: with GoalTest “any jug has 4”, BFS stops there.

Note: Two different goals give different answers

If the goal is exactly $(4, 4, 0)$, the shortest solution has 7 moves. If the goal is “any jug holds 4 litres” (the slide’s exercise), the state $(1, 4, 3)$ satisfies it after only 6 moves. **The GoalTest is part of the problem definition**, and changing it changes the solution.

2.1.4 The complete state space

Starting from $(8, 0, 0)$, exactly **16** states are reachable. The slide draws them on a ring, with $(0, 5, 3)$ in the middle. The figure below is generated from the pour rule itself: bold edges are the two solution paths; grey edges are all the other moves; a double arrowhead marks a move that can be undone by a single pour.



Definition: Reversible vs irreversible moves

A move $N \rightarrow N'$ is **reversible** if $N \in \text{MOVEGEN}(N')$, i.e. one pour takes you straight back. Example: $(8, 0, 0) \leftrightarrow (3, 5, 0)$. A move is **irreversible** if no single pour undoes it. Example: $(3, 2, 3) \rightarrow (0, 5, 3)$: from $(0, 5, 3)$ you can only reach $(5, 0, 3)$ or $(3, 5, 0)$, not $(3, 2, 3)$.

Intuition: Why some pours cannot be undone

A pour always ends with the source *empty* or the destination *full* — it cannot stop anywhere in between. The move $(3, 2, 3) \rightarrow (0, 5, 3)$ pours the 3 litres in the 8-litre jug into the 5-litre jug (which held 2), filling it. To undo it you would pour back from the 5-litre jug into the 8-litre jug and stop after exactly 3 litres; but a pour cannot stop there, so all 5 litres go and you reach $(5, 0, 3)$ instead. Irreversible moves are why the state space is a *directed* graph: 58 directed moves among 16 states, of which 24 have no single-pour reverse.

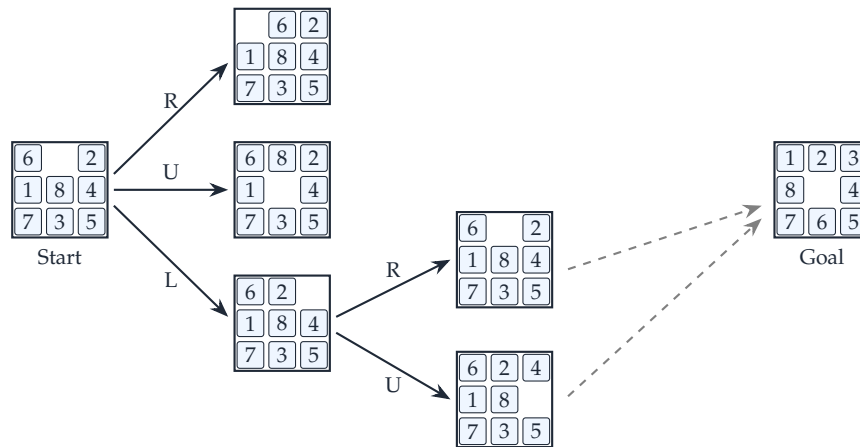
Note: What the slide leaves out

“We do not consider those cases where one can throw away some water.” Allowing emptying onto the ground (or filling from a tap) adds moves and changes the state space: the invariant $a + b + c = 8$ no longer holds, and more states become reachable.

2.2 The Eight-puzzle

Definition: Eight-puzzle

Eight numbered tiles on a 3×3 grid with one empty cell. A tile adjacent to the empty cell can slide into it. A move is labelled **R** if a tile moves right, and likewise **U** (up), **D** (down) and **L** (left). *Planning* algorithms use such **named moves**.

**Note: Reading the move labels**

From the start state the blank is in the top middle. Three tiles can move into it: 6 moves **Right**, 8 moves **Up**, 2 moves **Left**. After **L**, the blank is top right; now 4 can move **U**, and 2 can move back **R** — which simply undoes the previous move. Every 8-puzzle move is reversible, so the state space is an undirected graph (unlike the water jugs).

Formula / Derivation: Size of the Eight-puzzle state space

There are $9! = 362,880$ ways to place 8 tiles and a blank. A parity argument (each move is a transposition of the blank with a tile; the permutation parity of the tiles combined with the blank's row is invariant) splits these into two halves that cannot reach each other:

$$\text{reachable from any given state} = \frac{9!}{2} = 181,440.$$

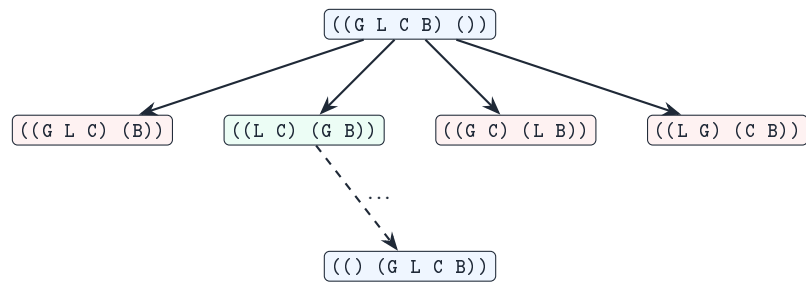
Branching factor: 2 (blank in a corner), 3 (blank on an edge) or 4 (blank in the centre). For the slide's start and goal, a breadth-first search (computed for these notes) finds that the shortest solution has **13 moves**. The 15-puzzle has $16!/2 \approx 1.05 \times 10^{13}$ reachable states and the 24-puzzle $25!/2 \approx 7.8 \times 10^{24}$ — the same problem, exploding combinatorially.

2.3 Man, Goat, Lion, Cabbage (MGLC)**Definition: The MGLC problem**

A man must take a lion, a goat and a cabbage across a river. His boat carries him and at most one of them. Only his presence stops the lion eating the goat and the goat eating the cabbage, so he can never leave the goat alone with the lion, nor the cabbage alone with the goat. How can he take them all across?

The lecture uses this problem to show that **one problem admits several representations**.

Representation	Start $\rightarrow$ goal	Comment
1. (Left-bank objects) (Right-bank objects)	$((G\ L\ C\ B)\ ()) \rightarrow (())\ (G\ L\ C\ B))$	Explicit and symmetric; redundant, because the right bank is just “everything not on the left”. B (the boat) marks where the man is.
2. (Left-bank objects)	$(G\ L\ C\ B) \rightarrow ()$	Half the size, same information.
3. (Objects on the boat's side, which bank the boat is on)	$(G\ L\ C\ Left) \rightarrow (G\ L\ C\ Right)$	Lists only what the man can take next; the man is implicit in every state. Trickier to read, but MoveGen is simpler: choose at most one object from the list.



First level of the MGLC state space in representation 1: the four candidate crossings (alone, with G, with L, with C). Only taking the goat (green) is safe; the others leave the goat with the lion or with the cabbage, unattended (red).

Exam trap: Generate, then filter

The slide's tree shows all four crossings as children of the start state. Three of them are *illegal* states. Your MoveGen must filter out unsafe states (or they must be rejected some other way); otherwise the search will happily let the lion eat the goat. Filtering inside MoveGen is the cleanest choice: the search algorithm then never sees an illegal state.

Algorithm: MoveGen and GoalTest for MGLC in representation 2 (answer to the slide exercise)

```

1: function MOVEGEN(L)                                ▷ L = set of items on the left bank, M/B marks the man
2:   here  $\leftarrow L$  if M  $\in L$  else {M,G,L,C} \ L      ▷ the bank the man is on
3:   result  $\leftarrow []$ 
4:   for each x  $\in \{\text{none}\} \cup (\text{here} \setminus \{M\})$  do                                ▷ cross alone, or with one item
5:     moved  $\leftarrow \{M\} \cup \{x\}$ 
6:     L'  $\leftarrow L \setminus \text{moved}$  if M  $\in L$  else L  $\cup$  moved
7:     if SAFE(L') and SAFE({M,G,L,C} \ L') then
8:       result  $\leftarrow L' : \text{result}$ 
9:     end if
10:  end for
11:  return result
12: end function
13: function SAFE(bank)
14:  return M  $\in \text{bank}$  or not ({G,L}  $\subseteq \text{bank}$  or {G,C}  $\subseteq \text{bank}$ )
15: end function
16: function GOALTEST(L)
17:  return L =  $\emptyset$                                 ▷ everything is on the right bank
18: end function

```

Formula / Derivation: How many MGLC states, and the solution

Each of M, G, L, C is on one of two banks: $2^4 = 16$ configurations. Unsafe ones leave G with L, or G with C, without M. On the left bank these are {G,L}, {G,C}, {G,L,C} (man away), and symmetrically on the right, which removes 6 configurations:

$$16 - 6 = 10 \text{ safe states, all reachable.}$$

The shortest solution has **7 crossings**, and there are exactly **2** shortest solutions:

Crossing	Solution 1	Solution 2
1	take G across	take G across
2	return alone	return alone
3	take L across	take C across
4	bring G back	bring G back
5	take C across	take L across
6	return alone	return alone
7	take G across	take G across

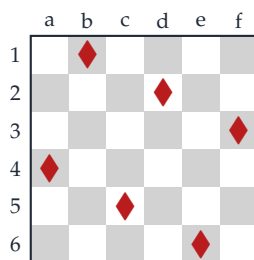
Left-bank states along solution 1: {M,G,L,C} → {L,C} → {M,L,C} → {C} → {M,G,C} → {G} → {M,G} → {}.

The key insight is step 4: bringing the goat *back*. A search algorithm finds this easily because it does not share the human bias against “undoing progress”.

2.4 The N-queens problem

Definition: N-queens

Place N queens on an $N \times N$ chess board so that no queen attacks another: no two share a row, a column or a diagonal. The slide’s instance is $N = 6$.



One of the 4 solutions of 6-queens (computed): queens at b1, d2, f3, a4, c5, e6.

In column-per-row form: (2, 4, 6, 1, 3, 5). The other three are its reflections and rotation: (3, 6, 2, 5, 1, 4), (4, 1, 5, 2, 6, 3), (5, 3, 1, 6, 4, 2).

Number of solutions for $N = 4, 5, 6, 7, 8$: 2, 10, 4, 40, 92 — not monotone!

Formula / Derivation: Why representation choice matters so much here

Different state spaces for 6-queens:

- Any 6 of the 36 squares: $\binom{36}{6} = 1,947,792$ candidate placements.
- One queen per row (rows cannot share queens anyway): $6^6 = 46,656$.
- One queen per row *and* per column (a permutation): $6! = 720$.
- Place row by row, rejecting any partial placement that is already attacked: only 153 nodes (partial placements, including the empty board and the 4 complete solutions) are ever generated — counted by running the search.

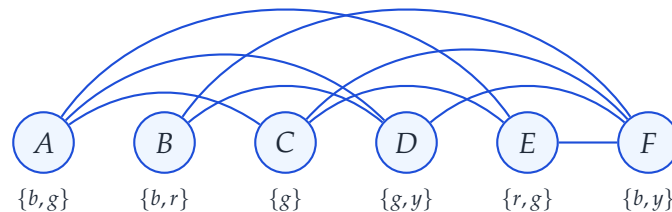
Building constraints into the representation shrinks the search space by orders of magnitude. N-queens is a **configuration problem**: we want the final board, not the order in which queens were placed.

2.5 A map colouring problem and its solutions

The second map colouring slide uses six regions A – F with domains

$$A\{b,g\}, \quad B\{b,r\}, \quad C\{g\}, \quad D\{g,y\}, \quad E\{r,g\}, \quad F\{b,y\}$$

(y = yellow). The constraint graph is drawn as arcs over a row of nodes; the order of the row is the order in which the algorithm assigns colours.



Borders read from the slide's arcs: A – C , A – D , A – E , B – D , B – F , C – E , C – F , D – F , E – F (9 constraints).

The slide shows four coloured maps, labelled with their colour sequences for (A, B, C, D, E, F) :

b, r, g, g, r, y b, r, g, g, r, b b, b, g, g, r, y b, r, g, y, r, b .

Formula / Derivation: Checking that there are exactly four solutions

Propagate forced values first:

1. $C \in \{g\}$ is forced: $C = g$.
2. A borders C , so $A \neq g \Rightarrow A = b$. E borders C , so $E \neq g \Rightarrow E = r$.
3. Remaining: $B \in \{b, r\}$, $D \in \{g, y\}$, $F \in \{b, y\}$ with $B \neq D$ (always true: the domains are disjoint), $B \neq F$ and $D \neq F$. The other constraints on D and F (A – D , C – F , E – F) are already satisfied by $A = b$, $C = g$, $E = r$. Note A and F are *not* adjacent, so $F = b$ is allowed.
4. Enumerate (D, F) with $D \neq F$: (g, b) , (g, y) , (y, b) [(y, y) is illegal]. Then $B \neq F$: for $F = b$, $B = r$ (1 way); for $F = y$, $B \in \{b, r\}$ (2 ways).

Total $1 + 2 + 1 = 4$: (b, r, g, g, r, b) , (b, b, g, g, r, y) , (b, r, g, g, r, y) , (b, r, g, y, r, b) — exactly the four maps on the slide (verified by computer enumeration of all $2^5 = 32$ assignments).

Note: Backtracking on this map

Assigning in the order $A, \dots, F$ and trying colours in the order b, g, r, y : $A = b$, $B = b$, $C = g$, $D = g$, $E = g$ fails ($= C$), $E = r$, $F = b$ fails ($= B$), $F = y$. Solution (b, b, g, g, r, y) with no backtracking. With the value order r, g, b, y , the search first tries $A = g$ and must **backtrack** three times (out of C twice, out of B once) before discovering that C 's only colour, g , is taken — a good reason to assign single-valued variables like C first.

2.6 The Traveling Salesman Problem

The slide calls TSP “the holy grail of computer science” and shows two tours of cities in India: a chaotic one (red, criss-crossing the country) and a good one (green, a smooth loop).

Definition: Traveling Salesman Problem (TSP)

Given n cities and the distance between every pair, find the **shortest tour** that visits each city exactly once and returns to the start.

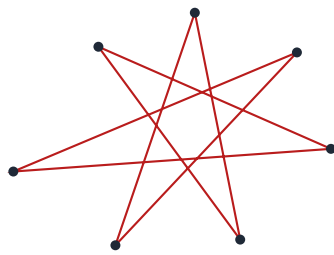
Formula / Derivation: How many tours?

Fix the starting city (any tour can be rotated to start there). The remaining $n - 1$ cities can be visited in $(n - 1)!$ orders, and each tour is counted twice (once in each direction):

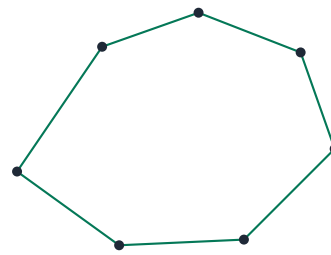
$$\text{\#tours} = \frac{(n - 1)!}{2}.$$

$n = 10$: 181,440. $n = 20$: $\approx 6.1 \times 10^{16}$. $n = 30$: $\approx 4.4 \times 10^{30}$.

Checking a billion tours per second, $n = 20$ takes about 2 years; $n = 30$ takes around 10^{14} years. TSP is **NP-hard**; no known algorithm solves it in polynomial time. It is an **optimisation** problem: every permutation is a valid tour, and the question is which is *best*.



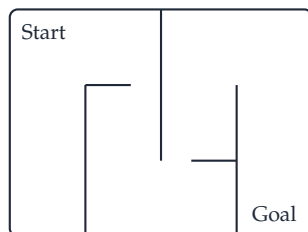
a bad tour (criss-crossing)



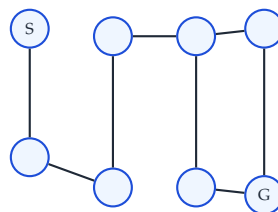
a good tour (no crossings)

Intuition: A useful fact about good tours

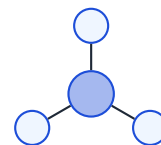
In the Euclidean TSP, an optimal tour never crosses itself: if two edges cross, swapping them (the “2-opt” move) always shortens the tour. That is why the green tour looks smooth. Local search methods in Week 3 use exactly such moves.

2.7 A maze: path finding as graph search

a maze of corridors



junctions $\rightarrow$ nodes, corridors $\rightarrow$ edges



what a search algorithm sees: only the current junction and its immediate options

Intuition: Bird's-eye view vs the algorithm's view

Looking down on a maze, you can often spot the route at once. A search algorithm has no such view: at each junction it sees only the corridors leaving that junction (MoveGen), and it must remember where it has been. This is the core difficulty of search, and it is why the lecture keeps using the maze metaphor.

Module recap

- Water jugs (8-5-3): states (a, b, c) with $a + b + c = 8$; pour until source empty or destination full; 16 reachable states; some moves irreversible $\Rightarrow$ directed graph; shortest route to $(4, 4, 0)$ is 7 pours.
- 8-puzzle: named moves R/U/L/D; $9!/2 = 181,440$ reachable states; all moves reversible.

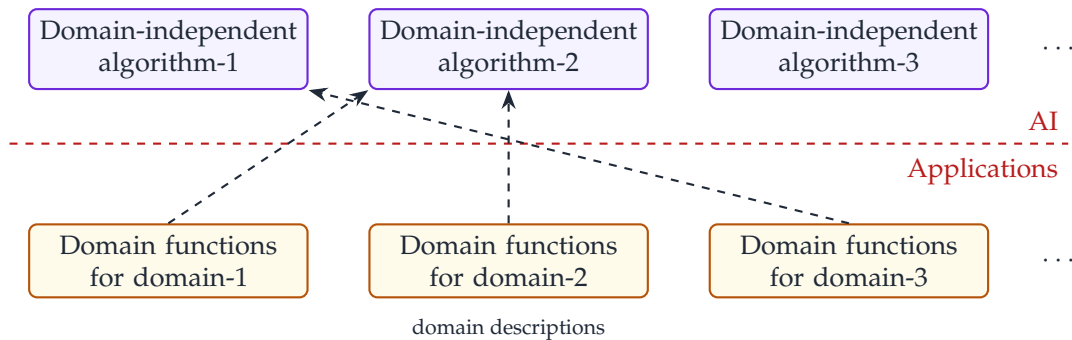
- MGLC: several representations of the same problem; 10 safe states; 7 crossings, 2 solutions (the goat goes back).
- N-queens and map colouring are *configuration* problems; TSP is an *optimisation* problem with $(n - 1)!/2$ tours.
- A maze shows what a search algorithm sees: only local options at each junction.

Module 3 General Search Algorithms

3.1 Domain-independent problem solving

Definition: The separation of concerns

Develop general problem-solving algorithms in a **domain-independent** form. For each domain, the user creates a **domain description** (the domain functions MoveGen and GoalTest, plus a representation of states) and **plugs it into** a suitable problem-solving algorithm.

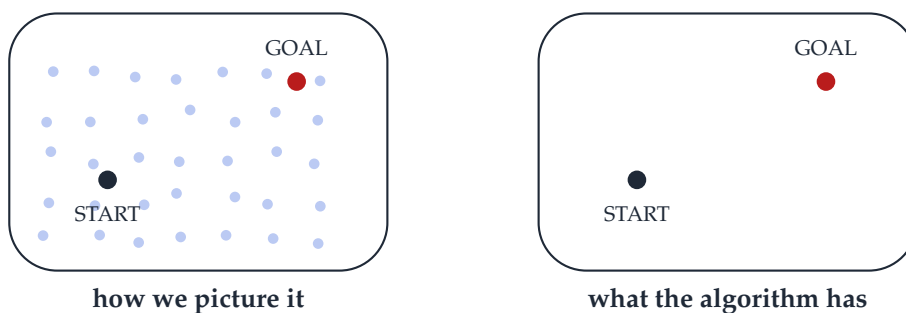


Intuition: A software-engineering view

This is an interface: the search algorithm is written against an abstract API (`move_gen(state) -> list[state]`, `goal_test(state) -> bool`), and each problem is an implementation of that API. Any algorithm can be combined with any domain. Changing the problem never requires touching the algorithm, and improving the algorithm helps every problem at once. Everything above the red line is “AI”; below it are “applications”.

3.2 The state space and its implicit nature

The slides draw the state space as a blob full of states, with one **START** state and one **GOAL** state. The next slide erases all the dots except these two: **the state space is implicit**. The algorithm does not have the picture; it has only the start state and two functions.



3.3 Moves: MoveGen

Definition: MoveGen

`MOVEGEN( $N$ )` is a **domain function** that returns the neighbours of node N . It is **provided by the user**. Applying it to N is called **expanding N** ; the returned nodes are its **children** or **successors**.

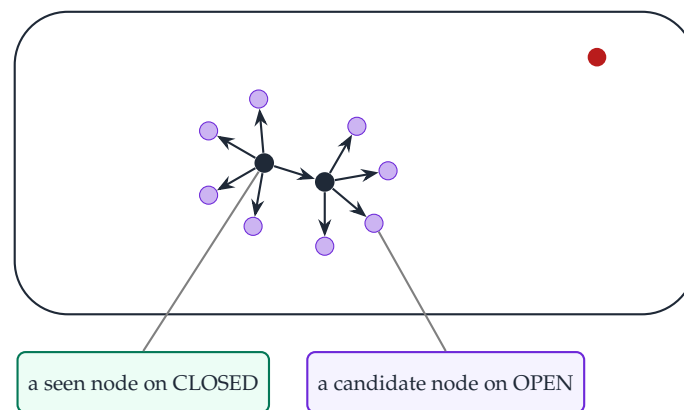
3.4 A search node is like a junction in a maze

In a maze you can only see the immediate options. Only when you choose one of them do the further options reveal themselves. Expanding a node is like walking to a junction and looking around.

3.5 OPEN and CLOSED

Definition: GoalTest, OPEN and CLOSED

- $\text{GOALTEST}(N)$: a **domain function**, provided by the user, that says whether N is a goal node.
- OPEN: the set of **candidate** nodes — generated but not yet inspected. It is the *frontier* of the search.
- CLOSED: the set of **seen** nodes — already picked, tested and expanded.



3.6 Search = treasure hunt: generate and test

Definition: Generate and test

Traverse the space by **generating** new nodes (using MoveGen) and **testing** each node (using GoalTest) to see whether it is the goal.

Intuition: The key question

The slide ends with the question that drives the whole course: **Which node to pick from OPEN?** Everything else — DFS, BFS, heuristic search, A* — is an answer to this one question. Blind (uninformed) methods answer it using only the *order* in which nodes were generated. Heuristic methods answer it using an *estimate* of how close each node is to the goal.

3.7 A tiny search problem

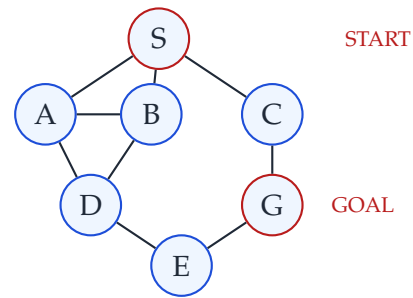
The MoveGen function

```

S → (A,B,C)
A → (S,B,D)
B → (S,A,D)
C → (S,G)
D → (A,B,E)
E → (D,G)
G → (C,E)

```

The state space



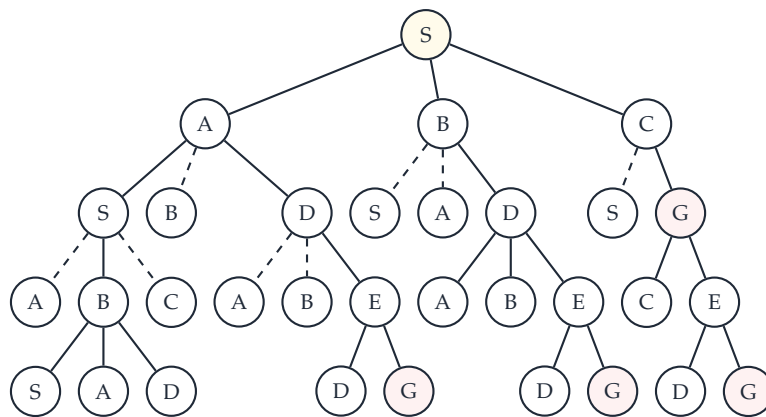
Note: Sets as lists

Neighbours form a *set*, but the slide represents them as a *list* returned by MoveGen. The list order matters: it decides which child a deterministic algorithm tries first.

3.8 The search space is a tree

Definition: Search tree vs state space

The **state space** is a graph: each state appears once. The **search tree** is what you get by starting at *S* and repeatedly expanding: each *path* from the root is a separate node. States reachable along several paths appear **many times**, and if the graph has cycles the tree is **infinite**.



The first levels of the (infinite) search tree for the tiny problem. G appears several times, at depths 2, 4 and more. Dashed branches are cut short for space.

Formula / Derivation: How fast does the search tree grow?

In the tiny problem every node has 2 or 3 neighbours. If every node had exactly b neighbours, the tree would have b^k nodes at depth k and

$$1 + b + b^2 + \dots + b^k = \frac{b^{k+1} - 1}{b - 1}$$

nodes up to depth k (geometric series: let $T = \sum_{i=0}^k b^i$; then $bT - T = b^{k+1} - 1$). The state space has 7 states, yet with $b = 3$ the tree has $3^{10} = 59,049$ nodes at depth 10. **The same few states are revisited over and over.**

3.9 SimpleSearch1

Algorithm: SimpleSearch1 (from the slide)

```

1: OPEN  $\leftarrow \{S\}$ 
2: while OPEN is not empty do
3:   pick some node  $N$  from OPEN
4:   OPEN  $\leftarrow$  OPEN  $- \{N\}$ 
5:   if GOALTEST( $N$ ) = TRUE then
6:     return  $N$ 
7:   else
8:     OPEN  $\leftarrow$  OPEN  $\cup$  MOVEGEN( $N$ )
9:   end if
10: end while
11: return FAILURE

```

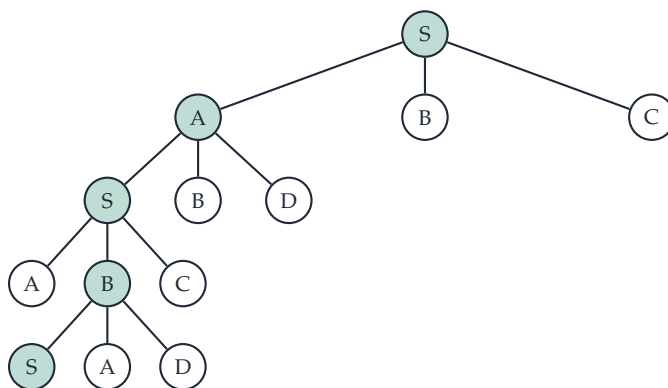
SimpleSearch1 picks a node N from OPEN and checks whether it is the goal. If yes, it returns N ; if not, it adds the children of N to OPEN.

3.9.1 SimpleSearch1 may loop

Nothing stops it from picking a node it has already seen. The slide shows one possible run (the picked node is underlined; its children replace it in the list):

$$(\underline{S}) \rightarrow (\underline{A}BC) \rightarrow (\underline{S}BDBC) \rightarrow (A\underline{B}CBDBC) \rightarrow (A\underline{S}ADCBDBC) \rightarrow \dots$$

The search went $S \rightarrow A \rightarrow S \rightarrow B \rightarrow S$: it keeps returning to S . OPEN grows forever and the algorithm may never terminate, even though G is only two moves away.



Green: nodes picked so far. The search walks $S, A, S, B, S, \dots$ round the cycle $S-A-B-S$.

Intuition: How do we prevent loops?

Remember where you have been. In a maze you might chalk a mark at every junction you pass through. That chalk mark is the CLOSED list.

3.10 SimpleSearch2: CLOSED, a repository of seen nodes

Algorithm: SimpleSearch2 (from the slide)

```

1: OPEN  $\leftarrow \{start\}$ 
2: CLOSED  $\leftarrow \{\}$ 
3: while OPEN is not empty do
4:   pick some node  $N$  from OPEN

```

```

5:  OPEN ← OPEN − {N}
6:  CLOSED ← CLOSED ∪ {N}           ▷ when a node is picked, add it to CLOSED
7:  if GOALTEST(N) = TRUE then
8:    return N
9:  else
10:   OPEN ← OPEN ∪ {MOVEGEN(N) − CLOSED}   ▷ do not add nodes already on
      CLOSED
11:  end if
12: end while
13: return FAILURE

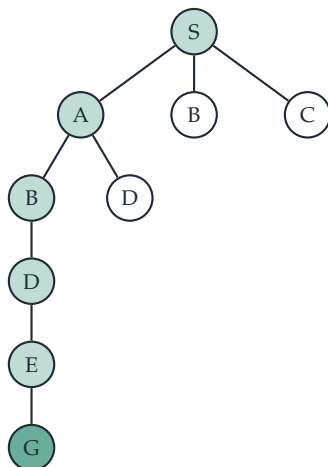
```

Formula / Derivation: Why CLOSED guarantees termination on a finite graph

Each iteration moves one node from OPEN to CLOSED, and a node on CLOSED is never added to OPEN again. So each of the $|V|$ states can be *picked and expanded* at most once (duplicates may sit on OPEN, but when a duplicate is picked its children are all filtered). The number of expansions is at most $|V|$ and every expansion adds at most b nodes to OPEN, so the loop runs at most $|V| + |V| \cdot b$ times. On a finite state space, SimpleSearch2 always terminates.

3.10.1 The search tree for SimpleSearch2

The slide's run (picking the head of OPEN, and putting new children at the front):



OPEN	CLOSED
(<u>S</u>)	()
(<u>A</u> B C)	(S)
(<u>B</u> D B C)	(A S)
(<u>D</u> D B C)	(B A S)
(<u>E</u> D B C)	(D B A S)
(<u>G</u> D B C)	(E D B A S)

Formula / Derivation: Tracing the table step by step

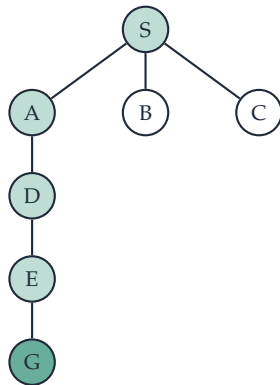
1. Pick S. Not goal. $\text{MOVEGEN}(S) - \text{CLOSED} = (A, B, C)$. $\text{OPEN} = (A, B, C)$.
2. Pick A. $\text{MOVEGEN}(A) = (S, B, D)$; S is on CLOSED, so add (B, D) at the front: $\text{OPEN} = (B, D, B, C)$. Note B now appears *twice*.
3. Pick B. $\text{MOVEGEN}(B) = (S, A, D)$; minus CLOSED $\{S, A, B\}$ leaves (D): $\text{OPEN} = (D, D, B, C)$.
4. Pick D. $\text{MOVEGEN}(D) = (A, B, E)$ minus CLOSED leaves (E): $\text{OPEN} = (E, D, B, C)$.
5. Pick E. (D, G) minus CLOSED leaves (G): $\text{OPEN} = (G, D, B, C)$.
6. Pick G. GoalTest succeeds; return G.

At termination OPEN still holds extra copies of D, B and C.

3.10.2 Adding only new nodes

A variation does not add nodes that are already on OPEN either:

$$\text{OPEN} \leftarrow \text{OPEN} \cup \{\text{MOVEGEN}(N) - \text{CLOSED} - \text{OPEN}\}.$$



OPEN	CLOSED
(S)	()
(A B C)	(S)
(D B C)	(A S)
(E B C)	(D A S)
(G B C)	(E D A S)

When A is expanded its children are (S, B, D): S is on CLOSED, B is already on OPEN, so only D is added. **Each node is added exactly once to the search tree**: the search tree becomes a spanning tree of the part of the graph explored.

Exam trap: “But is there a hidden cost here?”

The slide asks this and leaves it open. Two costs:

1. **Time.** Every new child must now be checked against OPEN as well as CLOSED. For breadth-first search OPEN can hold an exponential number of nodes, so a naive list search per child is expensive. (In practice one uses a hash set; the lectures use lists.)
2. **Which parent?** When a node is reached a second time, by a *different* path, the variation discards the new path. For DFS and BFS this does no harm, but when path cost matters (later: Dijkstra, A*), the second path might be *better*, and throwing it away loses the optimal solution. Algorithms that care about path cost must compare paths instead of blindly discarding duplicates.

3.11 Even when it succeeds, search does not return the path

SimpleSearch1 and SimpleSearch2 return the goal *node* G, not *how to get there*. For the tiny problem the answer “G” is useless: we knew G was the goal. What we need, in many problems, is the **path** to the goal node. The next module fixes this.

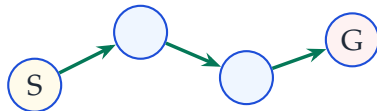
Module recap

- Domain-independent algorithms + user-supplied domain functions (MoveGen, GoalTest).
- OPEN holds candidates (the frontier); CLOSED holds seen nodes. Search = generate and test; the key question is which node to pick from OPEN.
- The search tree unfolds the state space; repeated states make it large or infinite.
- SimpleSearch1 can loop. SimpleSearch2 adds CLOSED and terminates on finite spaces. The variation that also skips nodes on OPEN adds each node to the tree exactly once, at the cost of extra checks and of discarding alternative parents.
- These searches return the goal node, not the path.

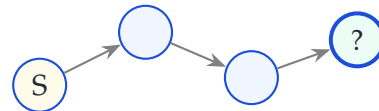
Module 4 Solutions as Paths: NodePairs, Determinism and Notation

4.1 Planning problems vs configuration problems

Planning problems	Configuration problems
The goal is <i>known or described</i> ; the path is sought.	A state satisfying a description is sought; how you got there does not matter.
<i>Examples:</i> river crossing problems, route finding, Rubik's cube, 8/15/24-puzzle, cooking a dish (and, by the same test, the water jug problem).	<i>Examples:</i> N-queens, a crossword puzzle, Sudoku, map colouring, SAT (Boolean satisfiability: find a truth assignment that makes a formula true).
Answer = a sequence of moves (a plan).	Answer = the final state.



Planning: G is known (e.g. the solved cube); the **green path** is the answer.



Configuration: we only know a *description* ("no two queens attack"); the **final state** is the answer.

Intuition: Why the distinction matters for algorithm design

For a configuration problem, SimpleSearch2 is already enough: returning the goal node *is* returning the answer. For a planning problem we must also return the path, so the search must remember how it reached each node. The water jug problem is a planning problem: "(4,4,0)" is not an answer, the sequence of pours is.

4.2 NodePairs: keeping track of parents

The state space is modelled as a graph with each node being a state. To reconstruct the path when the goal is found, we must record, during search, the **parent** of each node. Two options:

1. Store the **entire path** in each search node. Simple, but each node's storage grows with its depth, and paths share prefixes wastefully.
2. Store **node pairs** (*currentNode*, *parentNode*) in the search space. This is the elegant option the course uses. The start node's pair is (S, nil).

Formula / Derivation: Space cost of the two options

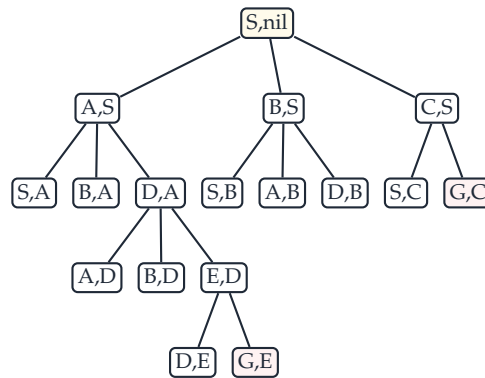
Suppose the search keeps n nodes, the average depth is $\bar{d}$, and a state name costs one unit.

$$\text{full paths: } \approx n(\bar{d} + 1) \text{ units,} \quad \text{node pairs: } 2n \text{ units.}$$

Node pairs cost a constant 2 units per node regardless of depth; the path is rebuilt only once, at the end, in $O(d)$ steps (plus the look-ups into CLOSED).

4.2.1 NodePairs in the search tree

Each node of the search tree for the tiny problem now carries its parent:



4.3 Reconstructing the path

The slide traces the variation that adds only new nodes, now with node pairs:

OPEN	CLOSED
((S,nil))	()
((A,S) (B,S) (C,S))	((S,nil))
((D,A) (B,S) (C,S))	((A,S) (S,nil))
((E,D) (B,S) (C,S))	((D,A) (A,S) (S,nil))
((G,E) (B,S) (C,S))	((E,D) (D,A) (A,S) (S,nil))

When (G, E) is picked and passes the goal test, follow the parent links backwards through CLOSED:

$$(G, E) \rightarrow (E, D) \rightarrow (D, A) \rightarrow (A, S) \rightarrow (S, \text{nil}).$$

Reading the first components in reverse gives the path $S \rightarrow A \rightarrow D \rightarrow E \rightarrow G$.

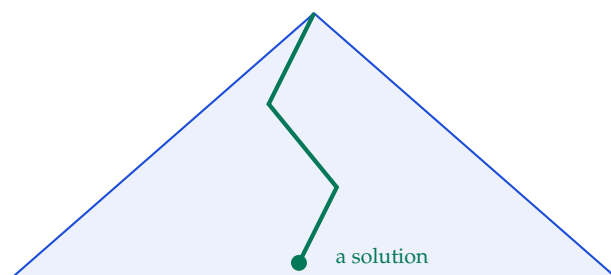


follow parents back to nil, then reverse: S, A, D, E, G

Note: Why the path is S-A-D-E-G, not the shorter S-C-G

This run always picks the head of OPEN and puts new children at the front, so it dives into A's subtree first (it is a depth-first search). S-C-G has only 2 moves, but C is never picked. Getting the shortest path requires a different choice of which node to pick — breadth-first search, Module 5.

4.4 A search program generates and explores a search tree



the state space explored by search, unfolded as a tree

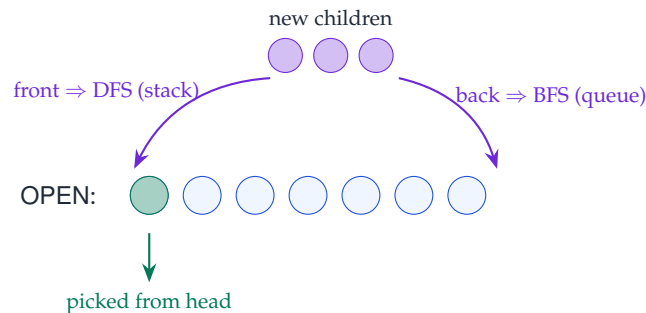
pick *some* node N from OPEN
 $\vdots$
 $\text{OPEN} \leftarrow \text{OPEN} \cup \text{MOVEGEN}(N)$
non-determinism

4.5 Deterministic search algorithms

“Pick *some* node” is **non-deterministic**: the algorithm does not say which. To get a concrete, reproducible algorithm:

- use **list** data structures instead of sets;
- replace “pick some node N from OPEN” with “pick the node at the **head** of OPEN”;
- instead of adding new nodes to OPEN as a set, **insert them at a specified place** in the list.

Where you insert them determines the behaviour of the search algorithm:



4.6 List notation

The algorithms on the slides use a small functional list notation (the same as in Haskell or Lisp).

#	Expression	Meaning
1	<code>[]</code>	the empty list
2–3	<code>[] is empty = TRUE</code> ; <code>[1] is empty = FALSE</code>	emptiness test
4–5	<code>LIST₂ ← ELEMENT : LIST₁</code> , <code>HEAD : TAIL</code>	“cons”: put an element in front of a list
6	<code>[1] = 1 : []</code>	a one-element list is an element consed onto the empty list
7–9	<code>head [1] = 1</code> , <code>tail [1] = []</code> , <code>(tail [1]) is empty = TRUE</code>	head = first element; tail = everything after it
10	<code>[3, 2, 1] = 3 : [2, 1] = 3 : 2 : [1] = 3 : 2 : 1 : []</code>	every list is built by repeated cons (: associates to the right)
11–14	<code>head [3, 2, 1] = 3</code> , <code>tail [3, 2, 1] = [2, 1]</code> , <code>head tail [3, 2, 1] = 2</code> , <code>head tail tail [3, 2, 1] = 1</code>	compose functions right to left
15–17	<code>LIST₃ = LIST₁ ++ LIST₂</code> ; <code>[] ++ [] = []</code> ; <code>L ++ [] = [] ++ L = L</code>	append (concatenate); <code>[]</code> is its identity
18–19	<code>[o, u, t, r, u, n] = [o, u, t] ++ [r, u, n]</code> ; <code>[r, u, n, o, u, t] = [r, u, n] ++ [o, u, t]</code>	order matters: <code>++</code> is not commutative
20	<code>[r, o, u, t] = (head [r, u, n]) : [o, u, t]</code>	mixing head and cons
21	<code>[n, u, t] = tail tail [r, u, n] ++ tail [o, u, t]</code>	<code>[n] ++ [u, t]</code>

Formula / Derivation: Cost of the list operations

For singly linked lists: **cons** ($x : L$), **head** and **tail** each take $O(1)$ time. **Append** $L_1 ++ L_2$ takes $O(|L_1|)$ time, because L_1 must be walked (and copied) to reach its end. Hence

$$\underbrace{\text{newPairs} ++ \text{tail OPEN}}_{\text{DFS: } O(b)} \quad \text{vs} \quad \underbrace{\text{tail OPEN} ++ \text{newPairs}}_{\text{BFS: } O(|\text{OPEN}|)}$$

where b is the number of new children. A real BFS implementation uses a queue with $O(1)$ enqueue (e.g. Python's `collections.deque`).

4.7 Tuple notation

#	Expression	Meaning
1	$(a, b) \leftarrow (101, 102)$	pattern matching: $a = 101, b = 102$
2–3	$\text{pair} \leftarrow (101, 102); (a, b) \leftarrow \text{pair}$	a tuple can be stored whole and taken apart later
4–5	$(a, _) \leftarrow \text{pair}; a \leftarrow \text{first pair}$	"_" is a wildcard: ignore that component
6–7	$(_, b) \leftarrow \text{pair}; b \leftarrow \text{second pair}$	
8–10	$(_, _, c) \leftarrow (101, \text{'AI SMPS'}, 4);$ $c \leftarrow \text{third } (\dots);$ $4 = \text{third } (101, \text{'AI SMPS'}, 4)$	tuples can have any length and mixed types
11	$101 = \text{head second } (1, [101, 102, 103], \text{null})$	the second component is a list; take its head
12	$[102, 103] =$ $\text{tail second } (1, [101, 102, 103], \text{null})$	
13	$(a, h : t, c) \leftarrow (1, [101, 102, 103], \text{null})$	nested pattern: $a = 1, h = 101,$ $t = [102, 103], c = \text{null}$

Note: Python equivalents, for your own implementations

$x:L \rightarrow [x] + L$; **head** $L \rightarrow L[0]$; **tail** $L \rightarrow L[1:]$; $L_1 ++ L_2 \rightarrow L_1 + L_2$; $(a, _) \leftarrow \text{pair} \rightarrow a, _ = \text{pair}$; $(a, h:t, c) \rightarrow a, (h, *t), c = \text{triple}$.

Module recap

- Planning problems want the path; configuration problems want the final state.
- Keep parents by storing *nodePairs* (node, parent); reconstruct the path by following parents back from the goal through CLOSED.
- Make search deterministic: lists instead of sets, pick from the head, insert new nodes at a chosen place — the place decides the algorithm.
- List notation: `[]`, `:` (cons), **head**, **tail**, `++` (append). Tuple notation: pattern matching with wildcards, **first**, **second**, **third**.

Module 5 Depth First and Breadth First Search

Both algorithms follow the same template: OPEN is a list of node pairs, the algorithm always picks the **head** of OPEN, and the *only* difference is where new pairs are inserted.

5.1 Depth First Search: OPEN is a stack

Algorithm: DFS(S) — from the slide

```

1: OPEN  $\leftarrow (S, \text{null}) : []$ 
2: CLOSED  $\leftarrow$  empty list
3: while OPEN is not empty do
4:   nodePair  $\leftarrow$  head OPEN
5:   (N, _)  $\leftarrow$  nodePair
6:   if GOALTEST(N) = TRUE then
7:     return RECONSTRUCTPATH(nodePair, CLOSED)
8:   else CLOSED  $\leftarrow$  nodePair : CLOSED
9:     children  $\leftarrow$  MOVEGEN(N)
10:    newNodes  $\leftarrow$  REMOVESEEN(children, OPEN, CLOSED)
11:    newPairs  $\leftarrow$  MAKEPAIRS(newNodes, N)
12:    OPEN  $\leftarrow$  newPairs ++ (tail OPEN) ▷ new nodes at the front
13:  end if
14: end while
15: return empty list

```

Note: Reading it line by line

Line 1 puts the start with no parent on OPEN. Lines 4–5 look at the head without removing it yet. If it is the goal, line 7 rebuilds the path. Otherwise line 8 records the pair on CLOSED; lines 9–11 generate children, drop any already seen, and pair each with its parent *N*. Line 12 is where DFS and BFS differ: DFS removes the head (**tail** OPEN) and puts the new pairs in *front* of the rest. Returning the empty list signals failure.

5.2 Ancillary functions

Algorithm: RemoveSeen(nodeList, OPEN, CLOSED) — removes nodes already in OPEN or CLOSED

```

1: if nodeList is empty then
2:   return empty list
3: else node  $\leftarrow$  head nodeList
4:   if OCCURSIN(node, OPEN) or OCCURSIN(node, CLOSED) then
5:     return REMOVESEEN(tail nodeList, OPEN, CLOSED)
6:   else return node : REMOVESEEN(tail nodeList, OPEN, CLOSED)
7:   end if
8: end if

```

Algorithm: OccursIn(*node*, *nodePairs*) — is *node* the first element of some pair?

```

1: if nodePairs is empty then
2:   return FALSE
3: else if node = first head nodePairs then
4:   return TRUE

```

```

5: else return OCCURSIN(node, tail nodePairs)
6: end if

```

Algorithm: MakePairs(*nodeList*, *parent*) — pair every node with the common parent

```

1: if nodeList is empty then
2:   return empty list
3: else return (head nodeList, parent) : MAKEPAIRS(tail nodeList, parent)
4: end if

```

Example: MAKEPAIRS([A, B, C, D], S) = [(A, S), (B, S), (C, S), (D, S)].

Algorithm: ReconstructPath(*nodePair*, CLOSED) — trace back through parent links

```

1: function SKIPTO(parent, nodePairs)
2:   if parent = first head nodePairs then
3:     return nodePairs
4:   else return SKIPTO(parent, tail nodePairs)
5:   end if
6: end function
7: (node, parent) ← nodePair
8: path ← node : []
9: while parent is not null do
10:  path ← parent : path
11:  CLOSED ← SKIPTO(parent, CLOSED)
12:  (__, parent) ← head CLOSED
13: end while
14: return path

```

Intuition: Why SkipTo only ever moves forward

CLOSED is built by consing, so the most recently closed pair is at the head. A node is always closed *after* its parent, so its parent's pair lies *further down* the list. Walking back from the goal, each successive ancestor is deeper still. SkipTo therefore never needs to restart from the top: the whole reconstruction is one left-to-right sweep of CLOSED, costing $O(|\text{CLOSED}|)$. Because RemoveSeen guarantees each node is closed at most once, the pair found for each ancestor is unique.

Formula / Derivation: Cost of RemoveSeen

OccursIn scans a list linearly, so checking one child costs $O(|\text{OPEN}| + |\text{CLOSED}|)$, and one expansion with b children costs $O(b(|\text{OPEN}| + |\text{CLOSED}|))$. Over a whole search that visits n nodes this is $O(bn^2)$ in the worst case — quadratic. Replacing the lists by a hash set of seen states brings each check to $O(1)$ expected and the total to $O(bn)$. The lecture uses lists for clarity, not efficiency.

5.3 Breadth First Search: OPEN is a queue

Algorithm: BFS(S) — from the slide

```

1: OPEN ← (S, null) : []
2: CLOSED ← empty list
3: while OPEN is not empty do
4:   nodePair ← head OPEN
5:   (N, _) ← nodePair
6:   if GOALTEST(N) = TRUE then

```

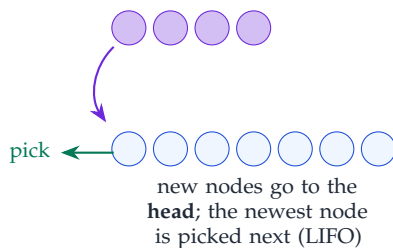
```

7:   return RECONSTRUCTPATH(nodePair, CLOSED)
8: else CLOSED  $\leftarrow$  nodePair : CLOSED
9:   children  $\leftarrow$  MOVEGEN(N)
10:  newNodes  $\leftarrow$  REMOVESEEN(children, OPEN, CLOSED)
11:  newPairs  $\leftarrow$  MAKEPAIRS(newNodes, N)
12:  OPEN  $\leftarrow$  (tail OPEN) ++ newPairs ▷ new nodes at the back
13: end if
14: end while
15: return empty list

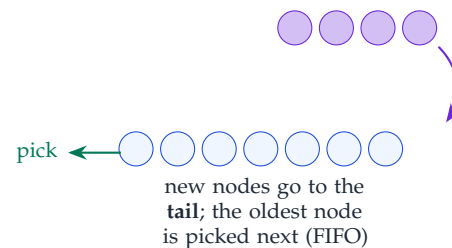
```

5.4 Stack vs queue

DFS: OPEN = STACK



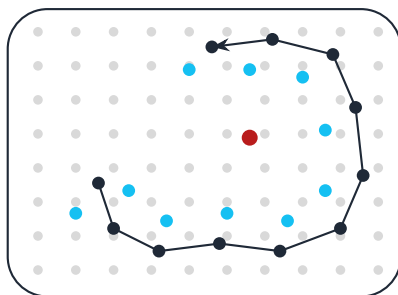
BFS: OPEN = QUEUE



Slide erratum: Stack vs. Queue slide

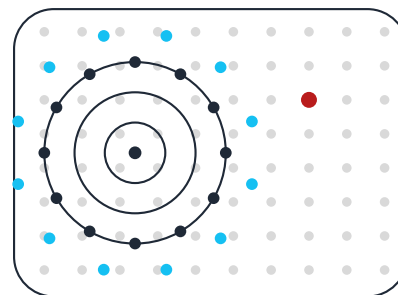
The slide's text box for BFS says "BFS adds new candidates at the *head* of OPEN". That is a copy-paste slip: its own arrow (and the BFS pseudocode, line 12: (**tail** OPEN) ++ *newPairs*) puts them at the **tail**. DFS adds at the head (stack); BFS adds at the tail (queue); both *pick* from the head.

5.5 How the two searches look on a large state space



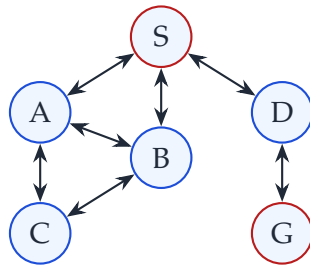
DFS: dives along one path

Black: expanded (CLOSED). Cyan: frontier (OPEN). Grey: not yet generated. Red: goal.
DFS runs deep and far along one route; BFS grows outward from the start, layer by layer.



BFS: expands in rings

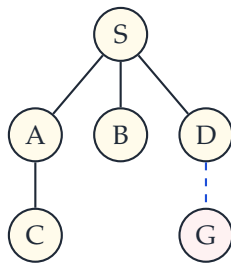
5.6 A tiny state space: three ways to handle duplicates



X	MOVEGEN(X)	GOALTEST(X)
S	A, B, D	False
A	C, B, S	False
B	S, A, C	False
C	B, A	False
D	S, G	False
G	D	True

5.6.1 Case 1: only new nodes are added to OPEN

This is the full RemoveSeen (skip nodes already on OPEN or CLOSED). Both DFS and BFS build the **same search tree**; only the order of exploration differs.



DFS

Pick	OPEN after the step	CLOSED after
	[(S,nil)]	[]
(S,nil)	[(A,S),(B,S),(D,S)]	[(S,nil)]
(A,S)	[(C,A),(B,S),(D,S)]	[(A,S),(S,nil)]
(C,A)	[(B,S),(D,S)]	[(C,A),...]
(B,S)	[(D,S)]	[(B,S),...]
(D,S)	[(G,D)]	[(D,S),(B,S),(C,A),(A,S),(S,nil)]
(G,D)	goal ⇒ path S, D, G	

BFS

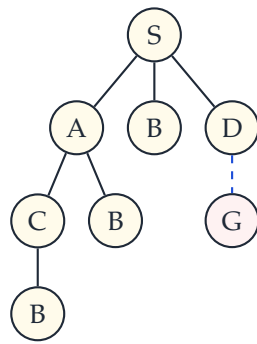
	[(S,nil)]	[]
(S,nil)	[(A,S),(B,S),(D,S)]	[(S,nil)]
(A,S)	[(B,S),(D,S),(C,A)]	[(A,S),(S,nil)]
(B,S)	[(D,S),(C,A)]	[(B,S),...]
(D,S)	[(C,A),(G,D)]	[(D,S),...]
(C,A)	[(G,D)]	[(C,A),(D,S),(B,S),(A,S),(S,nil)]
(G,D)	goal ⇒ path S, D, G	

Note: Details worth checking yourself

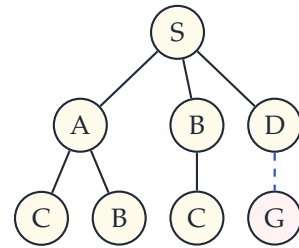
When A is expanded, its children are C, B, S. S is on CLOSED and B is on OPEN, so only C is new. When C is expanded (DFS), its children B and A are both seen, so nothing is added. Picking order: DFS S, A, C, B, D, G; BFS S, A, B, D, C, G. Both inspect 6 nodes here and return the same path, but for different reasons: DFS happened to finish A's subtree quickly.

5.6.2 Case 2: nodes already in CLOSED are pruned (but not those on OPEN)

Now a node already on OPEN can be added again, so trees differ:



DFS search tree

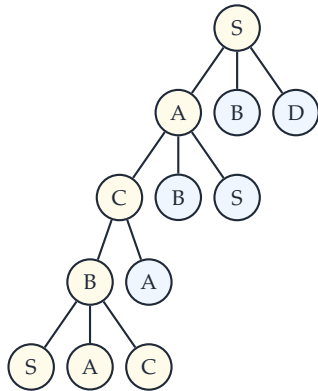


BFS search tree

Formula / Derivation: DFS trace for case 2

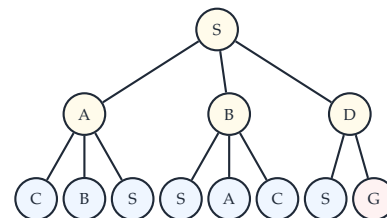
Pick	Children minus CLOSED	OPEN (node names only)
S	A, B, D	[A, B, D]
A	C, B (S closed)	[C, B, B, D]
C	B (A closed)	[B, B, B, D]
B	— (S, A, C closed)	[B, B, D]
B	—	[B, D]
B	—	[D]
D	G	[G]
G	goal	path S, D, G

B is picked three times: the duplicates sit on OPEN until picked, and each is expanded to nothing. Case 2 is correct but wasteful. (The slide draws each tree with B shown once per place it was added.)

5.6.3 Case 3: all new nodes are added (no duplicate check at all)

DFS: **infinite loop** S, A, C, B, S, A, C, B,

...



BFS: terminates with the **shortest path** S, D, G

Intuition: Why BFS survives without duplicate checks and DFS does not

DFS always extends the *newest* path. Following MoveGen order, A's first child is C, C's first child is B, B's first child is S, and S's first child is A again — a cycle it can never leave. BFS finishes every path of length k before any path of length $k+1$, so it cannot be trapped in a cycle: the goal at depth 2 is reached after all depth-1 paths, however many duplicates exist. The price is that BFS generates many duplicate nodes: here 8 nodes at depth 2 (where case 1 had only C and G), and before G is picked it also expands the seven depth-2 nodes queued ahead of it, generating a whole depth-3 layer. (The slide's tree omits D's child S.)

Duplicate handling	DFS	BFS
Case 1: skip nodes on OPEN or CLOSED	terminates; tree = spanning tree	terminates; shortest path
Case 2: skip nodes on CLOSED only	terminates; duplicates on OPEN	terminates; shortest path
Case 3: no check	may loop forever	terminates if a goal exists; shortest path; many duplicates

5.7 Running both on the tiny problem of Module 3

On the tiny problem of Module 3 (MoveGen: $S \rightarrow (A, B, C)$, $A \rightarrow (S, B, D)$, $B \rightarrow (S, A, D)$, $C \rightarrow (S, G)$, $D \rightarrow (A, B, E)$, $E \rightarrow (D, G)$, $G \rightarrow (C, E)$), with case 1 duplicate handling:

	DFS	BFS
OPEN sequence	[S] [A B C] [D B C] [E B C] [G B C]	[S] [A B C] [B C D] [C D] [D G] [G E]
Nodes picked	S, A, D, E, G	S, A, B, C, D, G
Path returned	S-A-D-E-G (4 moves)	S-C-G (2 moves, shortest)

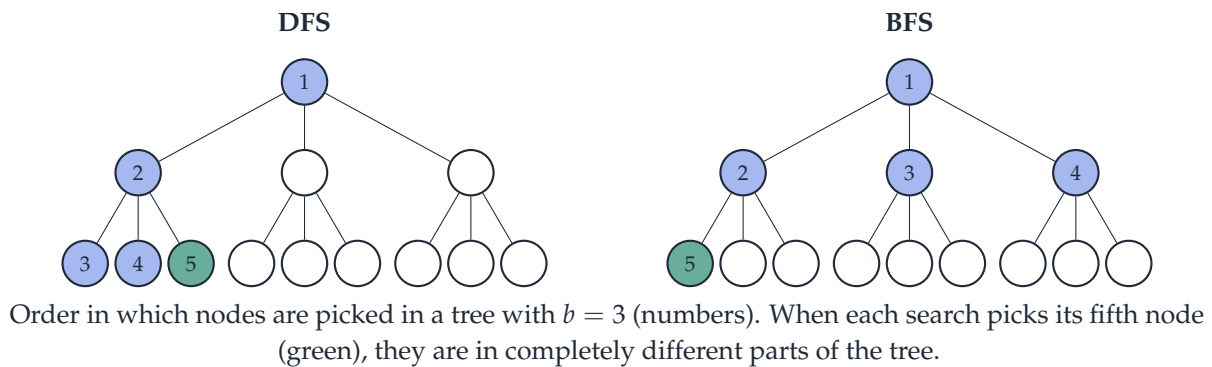
Module recap

- DFS: $OPEN \leftarrow newPairs ++ tail\ OPEN$ (stack). BFS: $OPEN \leftarrow tail\ OPEN ++ newPairs$ (queue). Both pick the head.
- Ancillary functions: RemoveSeen (uses OccursIn), MakePairs, ReconstructPath (uses SkipTo, one pass over CLOSED).
- Duplicate handling: skipping OPEN and CLOSED gives each node once; skipping CLOSED only is correct but wasteful; no check makes DFS loop on cycles but BFS still finds the shortest path.
- BFS returns the shortest path (fewest moves); DFS returns whatever path it hits first.

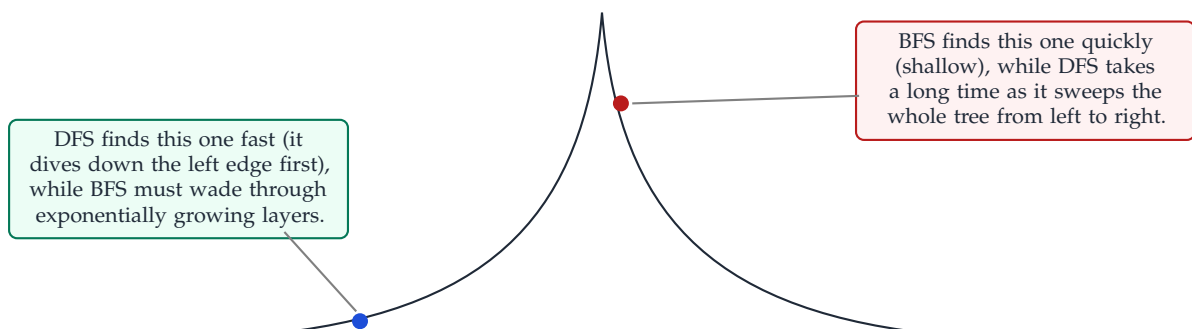
Module 6 Analysis of DFS and BFS

We compare the algorithms on four criteria: **time** (how many nodes are inspected), **space** (how large OPEN grows), **quality** of the solution (is the path shortest?) and **completeness** (is a solution guaranteed if one exists?).

6.1 Behaviour: the order of expansion



6.2 Running time depends on where the goal is



6.3 Time complexity

Definition: Assumptions for the analysis

The search tree has a constant **branching factor** b (every internal node has b children), and the goal occurs somewhere at depth d . The tree is searched only to depth d (so DFS does not wander below the goal's depth). Let N_{DFS} and N_{BFS} be the numbers of nodes **inspected** (picked and goal-tested).

Formula / Derivation: Nodes up to a depth: the geometric series

Level i of the tree has b^i nodes. The number of nodes at depths 0 to k is

$$T(k) = \sum_{i=0}^k b^i = \frac{b^{k+1} - 1}{b - 1} \quad (\text{nodes}).$$

Derivation: $bT(k) - T(k) = (b + b^2 + \dots + b^{k+1}) - (1 + b + \dots + b^k) = b^{k+1} - 1$.

Useful consequence: the last level dominates. $b^k / T(k) \rightarrow (b - 1) / b$ as k grows, so for $b = 10$ the bottom level alone holds about 90% of all the nodes.

6.3.1 Best case: goal at the left end of level d

- DFS dives straight down the left edge: it inspects exactly one node per level.

$$N_{\text{DFS}}^{\min} = d + 1.$$

- BFS must first inspect every node on levels $0, \dots, d-1$, then the first node of level d :

$$N_{\text{BFS}}^{\min} = T(d-1) + 1 = \frac{b^d - 1}{b - 1} + 1.$$

6.3.2 Worst case: goal at the right end of level d

Both inspect the entire tree down to depth d :

$$N_{\text{DFS}}^{\max} = N_{\text{BFS}}^{\max} = T(d) = \frac{b^{d+1} - 1}{b - 1}.$$

6.3.3 Average case

If the goal is equally likely to be anywhere on level d , the average is the mean of the best and worst cases (both counts grow linearly as the goal moves right along level d).

Formula / Derivation: Average number of nodes inspected

$$N_{\text{DFS}}^{\text{avg}} = \frac{(d+1) + \frac{b^{d+1} - 1}{b - 1}}{2} \approx \frac{b^{d+1}}{2(b-1)} \approx \frac{b^d}{2} \quad (\text{for large } b),$$

$$N_{\text{BFS}}^{\text{avg}} = \frac{\frac{b^d - 1}{b - 1} + 1 + \frac{b^{d+1} - 1}{b - 1}}{2} \approx \frac{b^d + b^{d+1}}{2(b-1)} = \frac{b^d(b+1)}{2(b-1)}.$$

Dividing, with the slide's approximation $N_{\text{DFS}} \approx b^d/2$:

$$N_{\text{BFS}} \approx N_{\text{DFS}} \cdot \frac{b+1}{b-1}$$

Average time complexity is exponential for both: $\Theta(b^d)$.

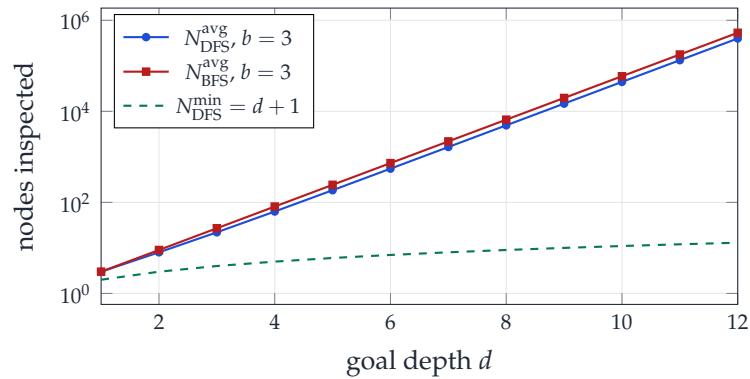
Note: A sharper constant

The step $\frac{b^{d+1}}{2(b-1)} \approx \frac{b^d}{2}$ drops a factor $\frac{b}{b-1}$. Keeping it gives the more accurate ratio

$$\frac{N_{\text{BFS}}}{N_{\text{DFS}}} \approx \frac{b^d(b+1)/(2(b-1))}{b^{d+1}/(2(b-1))} = \frac{b+1}{b}.$$

Both versions agree on the lesson: the two are within a small constant factor of each other, which tends to 1 as b grows. The table checks this against exact counts.

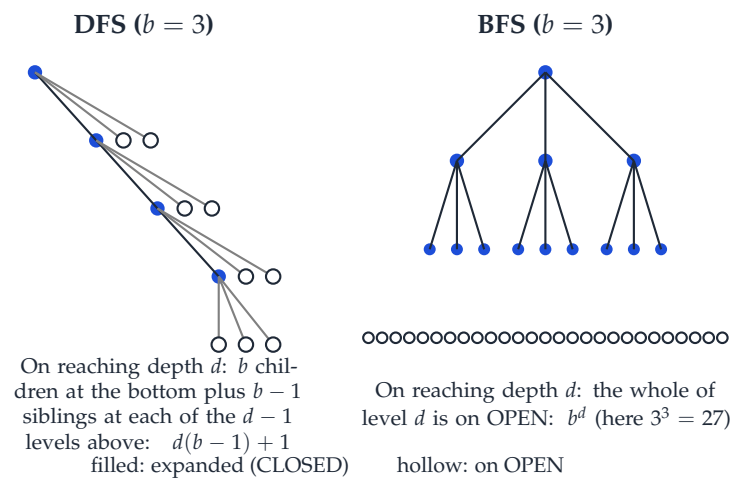
b	d	$T(d)$	$N_{\text{DFS}}^{\text{min}}$	$N_{\text{BFS}}^{\text{min}}$	$N_{\text{DFS}}^{\text{avg}}$	$N_{\text{BFS}}^{\text{avg}}$	exact ratio	$\frac{b+1}{b}$
2	10	2,047	11	1,024	1,029	1,535.5	1.49	1.50
3	8	9,841	9	3,281	4,925	6,561	1.33	1.33
5	5	3,906	6	782	1,956	2,344	1.20	1.20
10	5	111,111	6	11,112	55,558.5	61,111.5	1.10	1.10



On a log scale the average curves are straight lines of slope $\log 3$: exponential growth. They are almost on top of each other.

6.4 Space complexity: the size of OPEN

The number of nodes on the search frontier (OPEN) indicates the space requirement.



Formula / Derivation: Size of OPEN

DFS. When DFS first reaches depth d , each level $1, \dots, d - 1$ still holds the $b - 1$ unexplored siblings of the node on the current path, and level d holds all b children of the last expanded node:

$$|\text{OPEN}|_{\text{DFS}} = (d - 1)(b - 1) + b = d(b - 1) + 1 = O(bd) \quad \text{linear in } d.$$

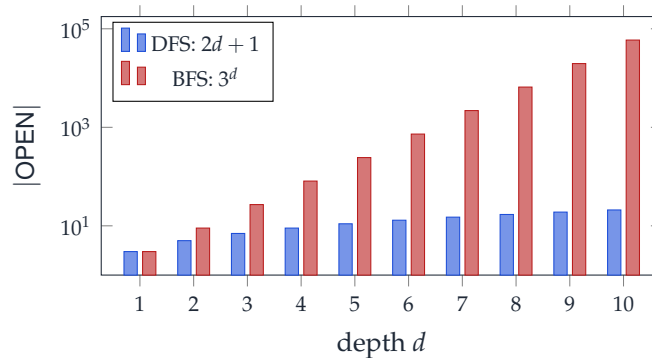
The slide's example: $b = 5, d = 5$: $5(5 - 1) + 1 = 21$ nodes. As DFS backtracks and moves right, the sibling lists shrink, so OPEN *decreases* from this maximum.

BFS. When BFS is about to inspect the first node of level d , OPEN holds *all* of level d :

$$|\text{OPEN}|_{\text{BFS}} = b^d = O(b^d) \quad \text{exponential in } d,$$

and it keeps growing (towards b^{d+1}) as level d nodes are expanded and their children appended.

Example: $b = 10, d = 10$: DFS needs $10 \cdot 9 + 1 = 91$ entries; BFS needs 10^{10} . At 100 bytes per node that is 9.1 KB versus 1 TB.



Frontier size for $b = 3$ (log scale). The DFS frontier grows linearly, the BFS frontier exponentially — the pink and yellow frontiers of the slide.

Exam trap: OPEN is not the whole story

The slides measure space by the size of OPEN. But DFS and BFS as written also keep CLOSED, and CLOSED eventually holds every node expanded — exponential for both on a large space. DFS's space is truly linear only if it does *not* keep a global CLOSED (it can keep just the current path to avoid cycles). This is exactly the design decision behind DFID in the next module.

6.5 Quality of solution and completeness

- **BFS finds the shortest path** (fewest moves). *Reason:* BFS inspects nodes in non-decreasing order of depth. If the shortest solution has length d , every node on level $< d$ is inspected before any node on level d , so no goal at depth $> d$ can be inspected before the one at depth d .
- **DFS has no guarantee:** it returns the first goal it runs into along its current path (e.g. S–A–D–E–G instead of S–C–G in Module 5).
- **BFS is complete:** if a solution path exists (and b is finite), BFS finds it, because it reaches every depth in turn — even in an infinite state space.
- **DFS is not complete for infinite search spaces:** it may follow an infinite path forever and never come back (e.g. an unbounded integer counter, or cycles with no CLOSED). On a finite graph with CLOSED, DFS is complete.

6.6 Depth First vs Breadth First: the summary table

	Depth First Search	Breadth First Search
Time	Exponential, $\Theta(b^d)$	Exponential, $\Theta(b^d)$
Space (size of OPEN)	Linear, $d(b - 1) + 1$	Exponential, b^d
Quality of solution	No guarantees	Shortest path
Completeness	Not for infinite search spaces	Guaranteed to terminate if a solution path exists

Intuition: The question that closes the analysis

Can we devise an algorithm that uses **linear space** for OPEN (like DFS) and **guarantees the shortest path** (like BFS)? Module 7 answers yes: depth-first iterative deepening.

Module recap

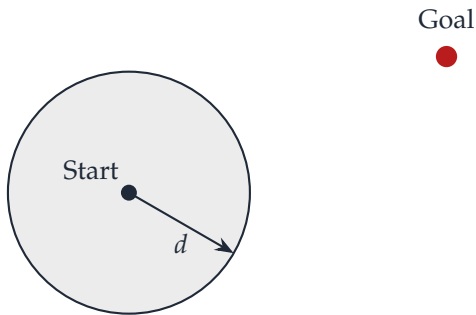
- Best case: DFS $d + 1$, BFS $\frac{b^d - 1}{b - 1} + 1$. Worst case: both $\frac{b^{d+1} - 1}{b - 1}$.
- Average: $N_{\text{DFS}} \approx b^d / 2$, $N_{\text{BFS}} \approx b^d (b + 1) / (2(b - 1))$; ratio $\approx \frac{b+1}{b-1}$ (slide) or more precisely $\frac{b+1}{b}$. Both exponential.
- Space (OPEN): DFS $d(b - 1) + 1$ (linear), BFS b^d (exponential).
- BFS: shortest path, complete. DFS: no guarantees, incomplete on infinite spaces.

Module 7 The Best of Both: Depth-First Iterative Deepening

7.1 Depth Bounded DFS (DBDFS)

Definition: Depth Bounded DFS

Do DFS, but never expand a node whose depth has reached a **depth bound** d . Nodes at depth d are goal-tested but their children are not generated.



Do DFS with a depth bound d :

- **linear space**
- **not complete** (the goal may lie outside the bound, as here)
- **does not guarantee the shortest path** (within the bound it is still DFS)

Algorithm: DB-DFS(S , depthBound) — store depth information in the search node

```

1: OPEN  $\leftarrow (S, \text{null}, 0) : []$  ▷ a triple, even if we still call it a nodePair
2: CLOSED  $\leftarrow$  empty list
3: while OPEN is not empty do
4:    $\text{nodePair} \leftarrow \text{head OPEN}$ 
5:    $(N, \_, \text{depth}) \leftarrow \text{nodePair}$ 
6:   if GOALTEST( $N$ ) = TRUE then
7:     return RECONSTRUCTPATH( $\text{nodePair}$ , CLOSED)
8:   else CLOSED  $\leftarrow \text{nodePair} : \text{CLOSED}$ 
9:     if  $\text{depth} < \text{depthBound}$  then ▷ search only within the depth bound
10:       $\text{children} \leftarrow \text{MOVEGEN}(N)$ 
11:       $\text{newNodes} \leftarrow \text{REMOVESEEN}(\text{children}, \text{OPEN}, \text{CLOSED})$ 
12:       $\text{newPairs} \leftarrow \text{MAKEPAIRS}(\text{newNodes}, N, \text{depth} + 1)$ 
13:      OPEN  $\leftarrow \text{newPairs} ++ \text{tail OPEN}$ 
14:    else OPEN  $\leftarrow \text{tail OPEN}$ 
15:    end if
16:  end if
17: end while
18: return empty list

```

Note: What changed from DFS

Three small changes: (1) every search node carries its depth, the start at depth 0; (2) Make-Pairs now builds triples ($\text{child}, N, \text{depth} + 1$); (3) a node at the bound is removed from OPEN without being expanded (line 14). Everything else — goal test, CLOSED, RemoveSeen, path reconstruction — is unchanged.

7.2 DBDFS-2: returning a count of nodes visited

Algorithm: DB-DFS-2(S , $depthBound$) — as DB-DFS, but also counts nodes generated

```

1:  $count \leftarrow 0$ 
2:  $OPEN \leftarrow (S, \text{null}, 0) : []$ ;  $CLOSED \leftarrow$  empty list
3: while  $OPEN$  is not empty do
4:    $nodePair \leftarrow \text{head } OPEN$ ;  $(N, \_, depth) \leftarrow nodePair$ 
5:   if  $\text{GOALTEST}(N) = \text{TRUE}$  then
6:     return ( $count$ ,  $\text{RECONSTRUCTPATH}(nodePair, CLOSED)$ )
7:   else  $CLOSED \leftarrow nodePair : CLOSED$ 
8:     if  $depth < depthBound$  then
9:        $children \leftarrow \text{MOVEGEN}(N)$ 
10:       $newNodes \leftarrow \text{REMOVESEEN}(children, OPEN, CLOSED)$ 
11:       $newPairs \leftarrow \text{MAKEPAIRS}(newNodes, N, depth + 1)$ 
12:       $OPEN \leftarrow newPairs ++ \text{tail } OPEN$ 
13:       $count \leftarrow count + \text{length } newPairs$ 
14:    else  $OPEN \leftarrow \text{tail } OPEN$ 
15:    end if
16:  end if
17: end while
18: return ( $count$ , empty list)

```

7.3 Depth First Iterative Deepening (DFID)

Algorithm: DFID(S) — a series of DB-DFS runs with increasing bounds

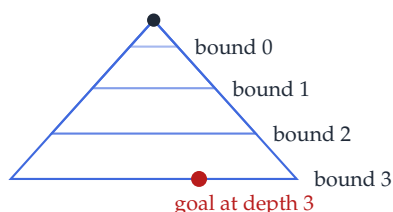
```

1:  $count \leftarrow -1$ 
2:  $path \leftarrow$  empty list
3:  $depthBound \leftarrow 0$ 
4: repeat
5:    $previousCount \leftarrow count$ 
6:    $(count, path) \leftarrow \text{DB-DFS-2}(S, depthBound)$ 
7:    $depthBound \leftarrow depthBound + 1$ 
8: until ( $path$  is not empty) or ( $previousCount = count$ )
9: return  $path$ 

```

Intuition: “Why count nodes?” — the answer to the slide’s question

If the state space is finite and has *no* goal, a plain “keep increasing the bound” loop would run forever. Once the bound exceeds the depth of the deepest reachable state, raising it further generates no new nodes, so DB-DFS-2 returns the **same count** twice in a row. That is proof that the whole reachable space has been explored without finding a goal: stop and report failure. The initial $count \leftarrow -1$ guarantees that the very first iteration (count 0 at bound 0) is never mistaken for “no change”.



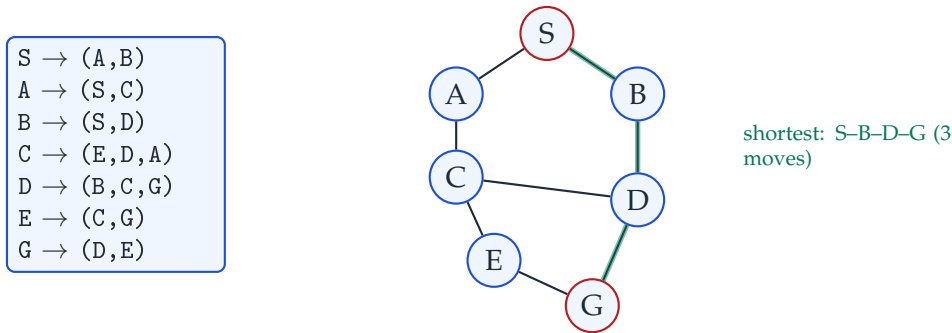
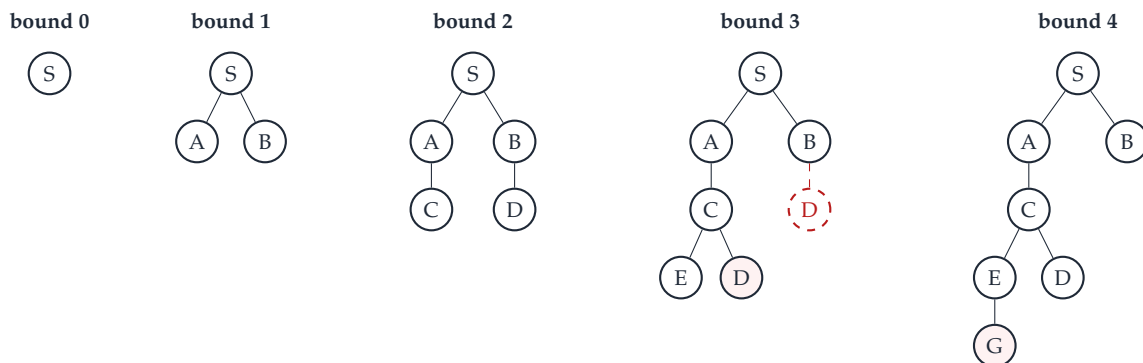
DFID does DB-DFS with bound 0, then 1, then 2, ... Each iteration is a fresh depth-first search from the start, exploring a bigger tree. The first iteration that reaches the goal finds it at the shallowest depth.

Definition: DFID as the slides finally state it

A series of depth-bounded **depth first** searches *without checking CLOSED* (thus requiring **linear space**) of increasing depth bound. When a path to the goal is found in some iteration, it is the **shortest path** — otherwise it would have been found in the previous iteration.

7.4 Does DFID return the shortest path? Careful with CLOSED

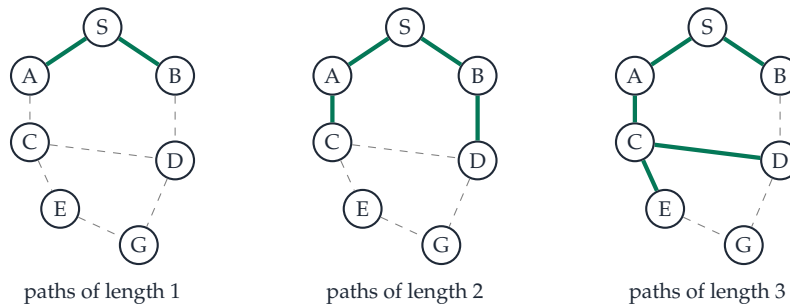
The slide credits this example to Siddharth Sagar, a student at IIT Dharwad, in 2019.

**7.4.1 DFID with CLOSED: iteration by iteration****Formula / Derivation: Why the depth-3 iteration misses the goal**

Bound 3, depth-first, with CLOSED:

1. Pick S (depth 0), expand: OPEN = $[A_1, B_1]$ (subscripts are depths).
2. Pick A_1 , expand: S is closed, so add C_2 : OPEN = $[C_2, B_1]$.
3. Pick C_2 , expand: E_3, D_3 (A closed): OPEN = $[E_3, D_3, B_1]$.
4. Pick E_3 : at the bound, not expanded (its child G at depth 4 is out of reach). Closed.
5. Pick D_3 : at the bound, not expanded (its child G would be at depth 4). **D is now on CLOSED.**
6. Pick B_1 , expand: S is closed, and **D is on CLOSED**, so nothing is added. The route S-B-D, which reaches D at depth 2 and G at depth 3, is **blocked**.

The iteration fails. Bound 4 then finds G via E: path S-A-C-E-G, length 4, not the shortest (3). Node counts returned by DB-DFS-2 (computed): bound 0: 0, bound 1: 2, bound 2: 4, bound 3: 5, bound 4: goal found.



With CLOSED, the length-3 iteration reaches D first via S-A-C-D and closes it, which blocks the length-2 path S-B-D and hence the shortest path S-B-D-G to the goal.

Slide erratum: DFID checking for loops with CLOSED

The slide's callout says the blocked shortest path is "A-B-D-G". The shortest path in this graph is **S-B-D-G**: A and B are not adjacent. The point being made is unchanged.

7.4.2 Without CLOSED, DFID finds the shortest path ...

If each iteration does *not* check CLOSED (at most, it avoids revisiting nodes on the *current path*, which prevents cycles), then within bound 3 the depth-first search explores S-A-C-E, S-A-C-D, and then S-B-D-G, and returns the **shortest path S-B-D-G** (computed: 8 nodes generated in that iteration).

7.4.3 ...but how does one reconstruct it?

The slide asks: "have we thrown the baby out with the bathwater?" Points to note:

- We still need the parent nodes to reconstruct the path.
- We can still store nodePairs in CLOSED and use them for reconstruction, but must be careful to choose the *right* nodePair, because a node can now be reached through **multiple parents**. (Here D has pairs (D,C) and (D,B); only (D,B) lies on the path found.)
- Perhaps we can store the **entire path** at each node? In DFID this is cheap: OPEN holds $O(bd)$ nodes, each with a path of length $\leq d$, so $O(bd^2)$ space — still polynomial. (A recursive implementation keeps the current path on the call stack for free.)
- For **configuration problems** the path is not needed at all. In fact the shortest "path" may have no importance except for time complexity.
- DFID enables **anytime** move generation in games like chess: the program deepens its search one ply at a time and, whenever the clock runs out, plays the best move from the deepest completed iteration. DFID was invented by chess programmers.

Algorithm: DFID as on the final slide (a series of DBDFSs)

```

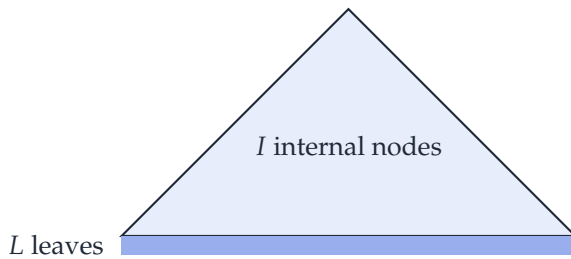
1: depthBound  $\leftarrow$  1
2: while TRUE do
3:   DEPTHBOUNDEDDFS(start, depthBound)
4:   depthBound  $\leftarrow$  depthBound + 1
5: end while

```

(Here the loop exits by returning from inside DepthBoundedDFS when a goal is found. The earlier version with *count* starts at bound 0 and also detects failure.)

7.5 “Is there a catch?” The cost of extra work

For every new layer of the search tree that DFID explores, it searches the **entire tree up to that layer all over again**. How much does this repetition cost?



In the last iteration, DFID inspects $L + I$ nodes (all internal nodes again, plus the new leaves). BFS inspects the internal nodes and leaves once, but the *new* work at this depth is L .

Formula / Derivation: Leaves and internal nodes of a full b -ary tree

Count edges two ways. Each node except the root has exactly one parent edge: #edges = $(L + I) - 1$. Each internal node has exactly b child edges: #edges = bI . Equating,

$$L + I - 1 = bI \implies L = (b - 1)I + 1.$$

Check with $b = 2$, depth 2: $I = 3$ (root and 2 children), $L = 4 = (2 - 1) \cdot 3 + 1$. ✓

Formula / Derivation: DFID vs BFS

Comparing the work at each new layer:

$$N_{\text{DFID}} = N_{\text{BFS}} \cdot \frac{L + I}{L} = N_{\text{BFS}} \left(1 + \frac{I}{(b - 1)I + 1} \right) \approx N_{\text{BFS}} \left(1 + \frac{1}{b - 1} \right) = N_{\text{BFS}} \cdot \frac{b}{b - 1}.$$

For large b , $\frac{b}{b-1} \rightarrow 1$: **the extra cost is not significant!**

Cross-check by summing all iterations. DFID with final bound d inspects $\sum_{k=0}^d T(k)$ nodes where $T(k) = \frac{b^{k+1}-1}{b-1}$; BFS (whole tree) inspects $T(d)$. Computed ratios:

b	d	BFS: $T(d)$	DFID: $\sum_k T(k)$	ratio	$\frac{b}{b-1}$
2	10	2,047	4,083	1.99	2.00
3	8	9,841	14,757	1.50	1.50
10	5	111,111	123,456	1.11	1.11

Even in the worst case, a binary tree, DFID does only about twice the work of BFS, while using linear instead of exponential space.

Intuition: Why re-doing work is cheap

In an exponentially growing tree, almost all nodes are on the *last* level (Module 6: about $\frac{b-1}{b}$ of them). Re-searching all the shallower levels therefore adds only a small fraction. It is like re-reading the first few pages of a book before every new chapter, when each chapter is ten times longer than everything before it.

Exam trap: The other catch

The analysis assumes a *tree*. On a graph with many cycles and no CLOSED, each iteration can revisit the same states along many different paths, so the number of paths up to depth d can greatly exceed the number of states. Checking the current path prevents infinite loops, but not this blow-up. DFID trades memory for repeated work; it shines when the state space is huge and tree-like (games, puzzles) and is poor when it is small and densely connected (use BFS with CLOSED there).

7.6 The final comparison

	DFS	BFS	DFID
Time	$\Theta(b^d)$	$\Theta(b^d)$	$\Theta(b^d), \approx \frac{b}{b-1} \times \text{BFS}$
Space (OPEN)	linear, $O(bd)$	exponential, $O(b^d)$	linear, $O(bd)$
Shortest path	no	yes	yes (without CLOSED)
Complete	not on infinite spaces	yes	yes

7.7 CombEx: combinatorial explosion

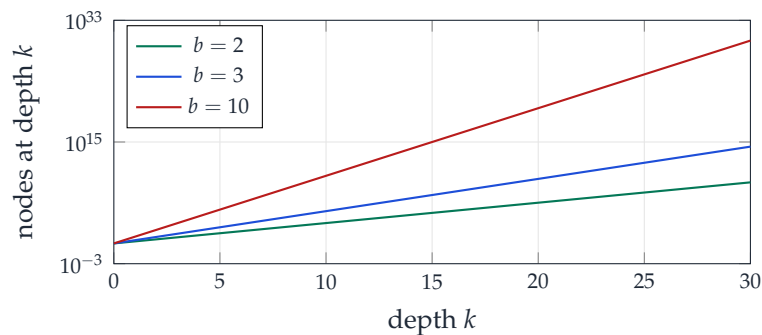
Definition: Combinatorial explosion

The monster that AI fights. For every node that fails the goal test, *all* its successors are added to the search tree. With b moves in every state the tree has b^k nodes at depth k .

Formula / Derivation: Even $b = 3$ grows fast

$$3^5 = 243, \quad 3^{10} = 59,049, \quad 3^{20} \approx 3.5 \times 10^9, \quad 3^{30} \approx 2.1 \times 10^{14}, \quad 3^{40} \approx 1.2 \times 10^{19}.$$

At a million nodes per second, depth 20 takes about an hour, depth 30 about 6.5 years, and depth 40 about 400,000 years. Faster computers only buy a few extra levels: a computer $3\times$ faster gets exactly **one** more level in the same time, since $\log_3 3 = 1$.



7.8 Blind search: same behaviour irrespective of the goal

The last two slides redraw the DFS and BFS pictures with the goal in several different places. **DFS always sweeps the same way and BFS always spirals out the same way, irrespective of where the goal node is.** Neither algorithm uses any information about the goal except the yes/no answer of GoalTest.

Definition: Blind / uninformed search

All three algorithms we studied — DFS, BFS and DFID — are **blind** or **uninformed**: the order in which they explore depends only on the structure of the space (and MoveGen's order), never on any sense of direction towards the goal.

Intuition: What comes next

To fight combinatorial explosion we need to answer the “which node to pick from OPEN?” question *intelligently*: prefer nodes that look closer to the goal. That requires a **heuristic function**

— knowledge about the domain — and it is the subject of Week 3 (heuristic search). Recall Elaine Rich's definition from Week 1: AI solves exponentially hard problems by exploiting knowledge about the problem domain.

Module recap

- DBDFS: DFS with a depth bound; linear space, incomplete, no shortest-path guarantee.
- DFID: DBDFS with bounds $0, 1, 2, \dots$; the count check detects when a finite space is exhausted.
- With CLOSED, DFID can miss the shortest path (S–B–D–G blocked because D was closed via a longer route). Without CLOSED it finds the shortest path but needs care to reconstruct it (store paths or use the recursion stack).
- Extra work: $L = (b - 1)I + 1 \Rightarrow N_{\text{DFID}} \approx N_{\text{BFS}} \cdot \frac{b}{b-1}$ — not significant.
- Combinatorial explosion defeats all blind search; heuristics (next week) are the remedy.

Practice: Test Yourself

Work each problem before reading its solution. Problems are tagged with the module they draw on. Every numerical answer below was checked by running code.

Part A — Numerical problems and algorithm traces

Problem N1

[Module 2] Water jugs, a different goal. With the 8-5-3 jugs, starting from (8,0,0), find the shortest sequence of pours that leaves exactly 1 litre in some jug.

Solution

Search breadth-first. Depth 1: (3,5,0), (5,0,3). Depth 2: (3,2,3), (0,5,3), (5,3,0). Depth 3: (6,2,0), (2,3,3). Depth 4 from (2,3,3): pour the 8-jug into the 5-jug ($t = \min(2, 5 - 3) = 2$) giving (0,5,3) — already seen — or pour 3-into-5 ($t = \min(3, 2) = 2$) giving (2,5,1). So:

$$(8,0,0) \xrightarrow{8 \rightarrow 3} (5,0,3) \xrightarrow{3 \rightarrow 5} (5,3,0) \xrightarrow{8 \rightarrow 3} (2,3,3) \xrightarrow{3 \rightarrow 5} (2,5,1).$$

4 pours (no state at depth ≤ 3 contains a 1). The left side of the ring reaches 1 litre later, at (1,5,2) after 5 pours.

Problem N2

[Module 2] Pour arithmetic. In state (6,0,2) list every legal pour with the amount $t = \min(x_i, \text{cap}_j - x_j)$ transferred and the resulting state. Which of the resulting moves are reversible?

Solution

Capacities (8,5,3).

Pour	t	Result	Reversible?
8 $\rightarrow$ 5	$\min(6, 5) = 5$	(1, 5, 2)	yes: 5 $\rightarrow$ 8 gives $\min(5, 7) = 5 \rightarrow (6, 0, 2)$
8 $\rightarrow$ 3	$\min(6, 1) = 1$	(5, 0, 3)	no: from (5, 0, 3), 3 $\rightarrow$ 8 moves 3, giving (8, 0, 0)
3 $\rightarrow$ 8	$\min(2, 2) = 2$	(8, 0, 0)	no: 8 $\rightarrow$ 3 moves 3, giving (5, 0, 3)
3 $\rightarrow$ 5	$\min(2, 5) = 2$	(6, 2, 0)	yes: 5 $\rightarrow$ 3 moves 2, giving (6, 0, 2)
5 $\rightarrow$ *	0	—	the 5-jug is empty

Four neighbours, matching the complete state space (Module 2).

Problem N3

[Module 5] DFS and BFS trace. MoveGen: $S \rightarrow (A, B)$, $A \rightarrow (S, C)$, $B \rightarrow (S, G)$, $C \rightarrow (A, D)$, $D \rightarrow (C, G)$, $G \rightarrow (B, D)$. Start S, goal G. Using the slide algorithms (RemoveSeen checks OPEN and CLOSED), give the sequence of OPEN and CLOSED for DFS and for BFS, and the path each returns.

Solution

DFS		BFS	
OPEN	CLOSED	OPEN	CLOSED
[S]	[]	[S]	[]
[A B]	[S]	[A B]	[S]
[C B]	[A S]	[B C]	[A S]
[D B]	[C A S]	[C G]	[B A S]
[G B]	[D C A S]	[G D]	[C B A S]
pick G: path S–A–C–D–G (4 moves)		pick G: path S–B–G (2 moves)	

DFS reconstructs via parents $(G, D) \rightarrow (D, C) \rightarrow (C, A) \rightarrow (A, S) \rightarrow (S, \text{nil})$; BFS via $(G, B) \rightarrow (B, S) \rightarrow (S, \text{nil})$. The graph is a cycle of length 6 and the goal is 2 steps one way and 4 the other: DFS happens to go the long way round.

Problem N4

[Module 5] Same graph, no duplicate checks. Run DFS on the graph of the previous problem with *no* checks against OPEN or CLOSED. Show the first five OPEN lists. What happens?

Solution

$[S] \rightarrow [A, B] \rightarrow [S, C, B] \rightarrow [A, B, C, B] \rightarrow [S, C, B, C, B] \rightarrow [A, B, C, B, C, B] \rightarrow \dots$

DFS alternates S, A, S, A, ... forever (A's first child is S, S's first child is A). OPEN grows by one node per two steps and G is never reached. BFS on the same inputs would still find S–B–G at depth 2.

Problem N5

[Module 5] Reconstructing a path. A search stops when it picks the pair (G, F) . At that moment $\text{CLOSED} = [(F, C), (E, B), (C, A), (D, A), (B, S), (A, S), (S, \text{nil})]$. Trace ReconstructPath, showing *path* and the head of CLOSED after each SkipTo.

Solution

Start: *path* = [G], parent = F.

1. *path* = [F, G]; SkipTo(F) leaves head $(F, C) \Rightarrow$ parent C.
2. *path* = [C, F, G]; SkipTo(C) skips $(F, C), (E, B)$; head $(C, A) \Rightarrow$ parent A.
3. *path* = [A, C, F, G]; SkipTo(A) skips $(C, A), (D, A), (B, S)$; head $(A, S) \Rightarrow$ parent S.
4. *path* = [S, A, C, F, G]; SkipTo(S): head $(S, \text{nil}) \Rightarrow$ parent nil; stop.

Path: **S–A–C–F–G**. Each SkipTo only moves forward: the whole reconstruction is one pass over CLOSED (7 pairs).

Problem N6

[Module 6] Time complexity numbers. A uniform tree has $b = 4$ and the goal at depth $d = 6$. Compute (a) the total number of nodes up to depth 6; (b) best-case counts for DFS and BFS; (c) average-case counts; (d) the slide's approximations $b^d/2$ and $b^d(b+1)/(2(b-1))$.

Solution

- (a) $T(6) = \frac{4^7-1}{3} = \frac{16383}{3} = \mathbf{5461}$.
 (b) $N_{\text{DFS}}^{\min} = d + 1 = \mathbf{7}$; $N_{\text{BFS}}^{\min} = \frac{4^6-1}{3} + 1 = 1365 + 1 = \mathbf{1366}$.
 (c) $N_{\text{DFS}}^{\text{avg}} = \frac{7+5461}{2} = \mathbf{2734}$; $N_{\text{BFS}}^{\text{avg}} = \frac{1366+5461}{2} = \mathbf{3413.5}$. Ratio $1.249 \approx \frac{b+1}{b} = 1.25$.
 (d) $4^6/2 = 2048$ (under-estimates DFS by the factor $\frac{b}{b-1} = \frac{4}{3}$; $2048 \times \frac{4}{3} = 2731$); $\frac{4096 \cdot 5}{2 \cdot 3} = 3413.3$ (very close to the exact 3413.5).

Problem N7

[Module 6] Space. For $b = 6$ and $d = 8$, how many nodes are on OPEN (a) for DFS when it first reaches depth 8, and (b) for BFS when it is about to inspect the first node at depth 8? (c) At 64 bytes per node, how much memory is that?

Solution

- (a) $d(b-1) + 1 = 8 \cdot 5 + 1 = \mathbf{41}$ nodes. (b) $b^d = 6^8 = \mathbf{1,679,616}$ nodes. (c) DFS: $41 \times 64 = 2624$ bytes ≈ 2.6 KB. BFS: $1,679,616 \times 64 \approx 1.07 \times 10^8$ bytes $\approx \mathbf{107}$ MB. Going to $d = 12$ multiplies the BFS figure by $6^4 = 1296$ (about 139 GB) and adds only $4 \times 5 = 20$ nodes to DFS.

Problem N8

[Module 7] Leaves and internal nodes. (a) For a full ternary tree ($b = 3$) of depth 3, count internal nodes I and leaves L and verify $L = (b-1)I + 1$. (b) Hence compute $(L+I)/L$ and compare with $\frac{b}{b-1}$.

Solution

- (a) Internal nodes are levels 0–2: $I = 1 + 3 + 9 = 13$. Leaves are level 3: $L = 27$. Check: $(3-1) \cdot 13 + 1 = 27$. ✓
 (b) $(L+I)/L = 40/27 \approx 1.48$, versus $b/(b-1) = 1.5$. The approximation $\frac{I}{L} \approx \frac{1}{b-1}$ drops only the “+1”.

Problem N9

[Module 7] Total cost of DFID. With $b = 4$ and the goal at depth 6 (worst case, rightmost), how many nodes does DFID inspect over all iterations, compared with BFS?

Solution

Iteration with bound k inspects $T(k) = \frac{4^{k+1}-1}{3}$ nodes: 1, 5, 21, 85, 341, 1365, 5461. Sum = **7279**. BFS inspects $T(6) = 5461$. Ratio $7279/5461 = \mathbf{1.33} = \frac{4}{3} = \frac{b}{b-1}$. DFID pays one third extra and in return needs only $6 \cdot 3 + 1 = 19$ OPEN entries instead of $4^6 = 4096$.

Problem N10

[Module 7] DFID with CLOSED: order matters. Repeat the Siddharth Sagar example, but with $\text{MoveGen}(S) = (B, A)$ instead of (A, B) . Does DFID with CLOSED now find the shortest path? What does this show?

Solution

Bound 3 (DB-DFS with CLOSED): pick S , $\text{OPEN} = [B_1, A_1]$. Pick B_1 : add D_2 . Pick D_2 : children B (closed), C_3 , G_3 : $\text{OPEN} = [C_3, G_3, A_1]$. Pick C_3 (at bound). Pick G_3 : **goal**. Path S – B – D – G ,

length 3 — shortest. (Counts per iteration, computed: 0, 2, 4, then success at bound 3.) So with CLOSED, DFID's correctness *depends on the order of MoveGen's output*: an algorithm that is only sometimes right is not correct. Dropping CLOSED (checking only the current path) makes it return a shortest path for every ordering.

Problem N11

[Module 3] Growth of the search tree. For the tiny problem of Module 3 (MoveGen $S \rightarrow (A,B,C)$, $A \rightarrow (S,B,D)$, $B \rightarrow (S,A,D)$, $C \rightarrow (S,G)$, $D \rightarrow (A,B,E)$, $E \rightarrow (D,G)$, $G \rightarrow (C,E)$), how many nodes does the full search tree (no duplicate checks) have at depths 0–4?

Solution

Expand level by level, summing out-degrees:
 depth 0: $[S] \rightarrow 1$. depth 1: $A, B, C \rightarrow 3$. depth 2: $3 + 3 + 2 = 8$. depth 3: the 8 nodes are S, B, D, S, A, D, S, G with out-degrees $3, 3, 3, 3, 3, 3, 3, 2$: **23**. depth 4: **62** (computed).
 Total to depth 4: $1 + 3 + 8 + 23 + 62 = 97$ tree nodes for a 7-state graph. Growth factor ≈ 2.7 per level, close to the average degree $18/7 \approx 2.6$.

Problem N12

[Module 2] Map colouring as state space search. For the six-region map ($A\{b, g\}$, $B\{b, r\}$, $C\{g\}$, $D\{g, y\}$, $E\{r, g\}$, $F\{b, y\}$; borders $A-C$, $A-D$, $A-E$, $B-D$, $B-F$, $C-E$, $C-F$, $D-F$, $E-F$), assign regions in the order $A, \dots, F$. (a) How many nodes does the full state space tree have? (b) If partial assignments that violate a constraint are discarded, how many consistent nodes are there at each depth?

Solution

(a) Nodes per depth = product of domain sizes so far: $1, 2, 4, 4, 8, 32$; total **67**.
 (b) Consistent partial assignments per depth (computed): $1, 2, 4, 2, 4, 4$; total **21**. At depth 3 half are lost ($A = g$ conflicts with $C = g$), and at depth 5 $E = g$ conflicts with C . The 4 nodes at depth 6 are the four solutions of the slide. Pruning early cuts the tree from 67 to 21 nodes.

Problem N13

[Module 2] TSP. (a) How many distinct tours are there for 8 cities? For 12? (b) If evaluating one tour takes $1 \mu s$, how long does brute force take for 12 cities, and for 20?

Solution

(a) $(n-1)!/2$: $n = 8$: $7!/2 = \mathbf{2520}$; $n = 12$: $11!/2 = \mathbf{19,958,400}$.
 (b) 12 cities: ≈ 20 seconds. 20 cities: $19!/2 \approx 6.08 \times 10^{16} \mu s \approx 6.1 \times 10^{10} s \approx \mathbf{1900 \text{ years}}$. Adding 8 cities multiplied the time by about 3×10^9 .

Problem N14

[Module 2] Eight-puzzle branching. What is the average branching factor of the 8-puzzle if the blank is equally likely to be in any of the 9 cells? Using $b \approx 2.67$ and a solution depth of 13 (the slide's instance), estimate the size of a blind search tree to depth 13 and compare it with the number of reachable states.

Solution

Corners (4 cells): 2 moves; edges (4 cells): 3; centre (1 cell): 4. Average = $\frac{4 \cdot 2 + 4 \cdot 3 + 1 \cdot 4}{9} = \frac{24}{9} \approx 2.67$.

$\sum_{k=0}^{13} 2.67^k \approx \frac{2.67^{14}}{1.67} \approx 5.6 \times 10^5$ tree nodes, about three times the 181,440 reachable states. The tree is larger than the graph because it revisits states (every move can be undone immediately). (A computed BFS with duplicate detection visited 3,656 states before reaching this particular goal.)

Problem N15

[Module 7] Combinatorial explosion. A computer inspects 10^7 nodes per second on a tree with $b = 3$. How deep can a full blind search go in one hour? How much deeper with a computer 100 times faster?

Solution

One hour = 3.6×10^{10} nodes. Need $3^d \lesssim 3.6 \times 10^{10}$: $d = \log_3(3.6 \times 10^{10}) = \frac{\ln(3.6 \times 10^{10})}{\ln 3} \approx \frac{24.3}{1.0986} \approx 22$.

100 times faster: extra depth = $\log_3 100 \approx 4.2$ levels, i.e. about 26. A hundredfold hardware speed-up buys only four more moves of look-ahead. This is why blind search needs heuristics.

Part B — Subjective questions with model answers**Question S1**

Explain what it means for a search algorithm to be domain-independent, and what the user must supply to use one.

Solution

A domain-independent algorithm is written without reference to any particular problem: it manipulates states only through two functions, MoveGen (the neighbours of a state) and GoalTest (is this state a goal?), and uses the start state as input. To apply it, the user supplies a domain description: a state representation, MoveGen and GoalTest. The same algorithm can then solve water jugs, the 8-puzzle, MGLC or route finding. This separates “AI” (general algorithms) from “applications” (domain descriptions) and means improvements to the algorithm help every domain.

Question S2

Why is the state space called an *implicit* graph? What consequences does this have for search algorithms?

Solution

The graph is never stored; it is defined by MoveGen: there is an edge from N to each state MoveGen returns. For realistic problems it is far too large to list (181,440 states for the 8-puzzle, astronomically more for chess). Consequences: the algorithm discovers the graph incrementally, as in a maze where only local options are visible; it must remember what it has seen (CLOSED) to avoid loops; it cannot know in advance how far the goal is; and its cost is measured in nodes generated or inspected rather than in graph size.

Question S3

Distinguish planning problems from configuration problems, with two examples each. How does the distinction affect what a search algorithm must return?

Solution

In planning problems the goal is known or described and the *path* to it is sought: river crossing, route finding, Rubik's cube, 8-puzzle, cooking a dish. In configuration problems a *state* satisfying a description is sought and the path does not matter: N-queens, Sudoku, map colouring, crosswords, SAT. For configuration problems returning the goal node suffices (SimpleSearch2 would do). For planning problems the algorithm must keep parent information (nodePairs) so it can reconstruct and return the path.

Question S4

Why can SimpleSearch1 fail to terminate, and how does SimpleSearch2 fix this? What is the "hidden cost" of the variation that also excludes nodes already on OPEN?

Solution

SimpleSearch1 never records visited nodes, so in a graph with cycles (S–A–S...) it can pick the same states forever, with OPEN growing without bound. SimpleSearch2 moves each picked node to CLOSED and never re-adds closed nodes to OPEN, so on a finite graph each state is expanded at most once and the search terminates. Excluding nodes already on OPEN too makes each node appear in the search tree exactly once. The hidden cost: every child must be checked against OPEN, which may be huge (exponential for BFS), and when a node is reached again via a different path, the new path is discarded — harmless for DFS/BFS but wrong when a later, cheaper path matters (A^*).

Question S5

Explain nodePairs and how ReconstructPath uses CLOSED. Why does SkipTo never need to go back to the start of CLOSED?

Solution

Each search node is stored as (node, parent), with the start as (S, nil). When the goal pair is picked, ReconstructPath starts the path with the goal, then repeatedly prepends the parent and looks up the parent's own pair in CLOSED to find the grandparent, until the parent is nil. CLOSED is built by consing, so later-closed nodes are nearer the head. Any node is closed after its parent, so as we walk from the goal back towards the start, each ancestor's pair lies further down the list than the previous one. SkipTo therefore only ever moves forward, and reconstruction is a single pass. RemoveSeen guarantees each node appears in CLOSED once, so the lookup is unambiguous.

Question S6

Explain how the choice of where to insert new nodes into OPEN turns the same template into DFS or BFS. Point out the error on the "Stack vs. Queue" slide.

Solution

Both algorithms always pick the head of OPEN. DFS puts new node pairs at the front ($newPairs ++ tail\ OPEN$): the newest node is explored next, so OPEN behaves as a stack (LIFO) and the search dives deep. BFS appends them at the back ($tail\ OPEN ++ newPairs$): the oldest node is explored next, OPEN is a queue (FIFO) and the search proceeds level by level. The slide's text says BFS adds new candidates "at the head of OPEN"; it should say at the tail, as the arrow and the BFS pseudocode show.

Question S7

Derive the best-case, worst-case and average numbers of nodes inspected by DFS and BFS, and state the conclusion about time complexity.

Solution

Assume branching factor b and the goal at depth d , tree searched to depth d . Nodes to depth k : $T(k) = \frac{b^{k+1}-1}{b-1}$. Best case (goal leftmost): DFS inspects one node per level, $d + 1$; BFS inspects all of levels $0..d - 1$ plus one, $\frac{b^d-1}{b-1} + 1$. Worst case (goal rightmost): both inspect $T(d)$. Averages are the means: $N_{DFS} \approx \frac{b^{d+1}}{2(b-1)} \approx \frac{b^d}{2}$, $N_{BFS} \approx \frac{b^d(b+1)}{2(b-1)}$, so $N_{BFS} \approx N_{DFS} \frac{b+1}{b-1}$ (the slide) or, keeping the dropped factor, $\frac{b+1}{b}$. Either way both are $\Theta(b^d)$: average time complexity is exponential for both, and they differ only by a small constant.

Question S8

Compare DFS and BFS on space, solution quality and completeness, explaining each property.

Solution

Space: DFS keeps only the siblings along the current path, $d(b - 1) + 1$ nodes on OPEN — linear. BFS keeps an entire level, b^d — exponential. **Quality:** BFS inspects nodes in order of depth, so the first goal found is at minimum depth: shortest path. DFS returns whatever goal it meets first on its current branch: no guarantee. **Completeness:** BFS reaches every depth in turn, so it finds a solution if one exists (finite b), even in infinite spaces. DFS can follow an infinite branch forever, so it is not complete on infinite spaces (on finite graphs with CLOSED it is). Caveat: both keep CLOSED, which grows with the number of nodes seen.

Question S9

Describe DFID. Why does it return the shortest path, and why does it use linear space?

Solution

DFID runs depth-bounded DFS with bound $0, 1, 2, \dots$ until a goal is found (or the node count stops changing, meaning a finite space was exhausted). Each iteration is a DFS, so OPEN holds at most $O(bd)$ nodes, and without a global CLOSED nothing else grows: linear space. The iteration with bound k explores every path of length $\leq k$. If the first success comes at bound k , no goal exists at depth $< k$ (otherwise iteration $k - 1$ would have found it), so the path found has length k , the minimum. This requires that each iteration explores *all* paths within the bound, which is why CLOSED must not be used.

Question S10

Using the Siddharth Sagar example, explain why DFID with CLOSED can fail to find the shortest path. What are the options for recovering the path without CLOSED?

Solution

In the graph S–A–C–(E,D), S–B–D, D–G, E–G, the bound-3 iteration goes S, A, C first and reaches D at depth 3, closing it. When it later expands B (depth 1), D is on CLOSED, so S–B–D (D at depth 2) and hence G at depth 3 are never generated. The next iteration finds S–A–C–E–G (length 4) instead of S–B–D–G (length 3). Without CLOSED the path can be recovered by: storing nodePairs and carefully choosing the right parent (nodes can have several); storing the full path in each node ($O(bd^2)$ space, still polynomial); or using recursion so the path lives on the call stack. For configuration problems the path is not needed at all.

Question S11

Show that DFID's repeated work is not significant. When might it still be a poor choice?

Solution

In a full b -ary tree $L = (b - 1)I + 1$ (count edges as $L + I - 1$ and as bI). At each new depth DFID inspects $L + I$ nodes where BFS's new work is L , so $N_{\text{DFID}} = N_{\text{BFS}} \frac{L+I}{L} \approx N_{\text{BFS}} \frac{b}{b-1}$: at most double (for $b = 2$) and close to 1 for large b . Most nodes are in the last level, so re-searching shallower levels is cheap. It can still be poor on small, densely connected graphs: without CLOSED, each iteration explores every *path* within the bound, and the number of paths can vastly exceed the number of states; BFS with CLOSED is better there.

Question S12

What is combinatorial explosion, and why are DFS, BFS and DFID called blind? What is the remedy?

Solution

Every node that fails the goal test contributes all its successors, so the tree has b^k nodes at depth k ; even $b = 3$ gives $3^{20} \approx 3.5 \times 10^9$ at depth 20. Faster hardware buys only $\log_b$ (speed-up) extra levels. DFS, BFS and DFID are blind (uninformed) because their exploration order depends only on the order nodes were generated, never on where the goal is: the slides' pictures look the same whatever the goal's location. The remedy is to use domain knowledge to choose which node to expand next — heuristic search, next week.

Part C — Multiple-choice questions

Answers and explanations follow the questions.

Q1. In state space search, the state space is:

- (a) stored as an adjacency matrix (b) an implicit graph defined by MoveGen (c) always a tree (d) always finite

Q2. Which two functions must the user supply to a domain-independent search algorithm?

- (a) Heuristic and Cost (b) MoveGen and GoalTest (c) Push and Pop (d) Head and Tail

Q3. In the 8-5-3 water jug problem, how many states are reachable from (8,0,0)?

(a) 8 (b) 12 (c) 16 (d) 24

Q4. The move $(3, 2, 3) \rightarrow (0, 5, 3)$ is:

(a) reversible (b) irreversible (c) illegal (d) a goal move

Q5. From $(5, 3, 0)$, pouring the 5-litre jug into the 3-litre jug gives:

(a) $(5, 0, 3)$ (b) $(2, 3, 3)$ (c) $(8, 0, 0)$ (d) $(5, 3, 0)$ (no change)

Q6. The number of states of the 8-puzzle reachable from a given state is:

(a) $9!$ (b) $9!/2$ (c) $8!$ (d) 2^9

Q7. In the 8-puzzle, a move labelled “R” means:

(a) the blank moves right (b) a tile moves right (c) rotate the board (d) restart

Q8. How many safe states does the MGLC problem have?

(a) 8 (b) 10 (c) 14 (d) 16

Q9. The minimum number of river crossings to solve MGLC is:

(a) 5 (b) 6 (c) 7 (d) 9

Q10. Which of these is a configuration problem?

(a) Rubik’s cube (b) route finding (c) N-queens (d) river crossing

Q11. The number of distinct tours in a TSP with n cities is:

(a) $n!$ (b) $(n - 1)!$ (c) $(n - 1)!/2$ (d) 2^n

Q12. SimpleSearch1 may fail to terminate because:

(a) OPEN is a stack (b) it does not remember visited nodes (c) GoalTest is expensive (d) MoveGen is non-deterministic

Q13. In SimpleSearch2, a node is added to CLOSED when it is:

(a) generated (b) picked from OPEN (c) found to be the goal only (d) never

Q14. SimpleSearch2 and its variation return:

(a) the path to the goal (b) the goal node only (c) the cost of the path (d) the size of CLOSED

Q15. The variation that adds only nodes not on OPEN or CLOSED ensures that:

(a) the shortest path is found (b) each node is added to the search tree exactly once (c) OPEN is always empty (d) GoalTest is never called twice

Q16. A nodePair stores:

(a) a node and its depth (b) a node and its parent (c) two sibling nodes (d) a node and its heuristic value

Q17. `head tail tail [3, 2, 1]` equals:

(a) 3 (b) 2 (c) 1 (d) [1]

Q18. `tail tail [r, u, n] ++ tail [o, u, t]` equals:

(a) $[n, u, t]$ (b) $[u, n, u, t]$ (c) $[n, o, u, t]$ (d) $[r, u, t]$

Q19. In $(a, h : t, c) \leftarrow (1, [101, 102, 103], \text{null})$, t is:

(a) 101 (b) $[101, 102, 103]$ (c) $[102, 103]$ (d) null

Q20. In DFS, line 12 of the algorithm is:

(a) `OPEN ← tail OPEN ++ newPairs` (b) `OPEN ← newPairs ++ tail OPEN` (c) `OPEN ← newPairs` (d) `OPEN ← OPEN ++ CLOSED`

- Q21.** OPEN in BFS behaves as a:
 (a) stack (b) queue (c) priority queue (d) set
- Q22.** RemoveSeen removes from the list of children those nodes that:
 (a) are goals (b) are already on OPEN or CLOSED (c) have no children (d) are deeper than the bound
- Q23.** MAKEPAIRS($[A, B], S$) returns:
 (a) $[(S, A), (S, B)]$ (b) $[(A, S), (B, S)]$ (c) $[(A, B)]$ (d) $[A, B, S]$
- Q24.** On the slide's tiny state space ($S \rightarrow A, B, D; A \rightarrow C, B, S; \dots$), when only new nodes are added:
 (a) DFS and BFS build different trees (b) DFS and BFS build the same tree but explore it in different orders (c) DFS loops (d) BFS fails
- Q25.** With no duplicate checks at all on a graph with cycles:
 (a) both DFS and BFS loop (b) DFS may loop forever; BFS still finds the shortest path (c) BFS loops; DFS succeeds (d) both find the shortest path
- Q26.** With branching factor b and goal at the leftmost node of depth d , DFS inspects:
 (a) d (b) $d + 1$ (c) b^d (d) $(b^{d+1} - 1) / (b - 1)$
- Q27.** In the same setting, BFS inspects:
 (a) $d + 1$ (b) $(b^d - 1) / (b - 1) + 1$ (c) b^{d+1} (d) bd
- Q28.** The slide's average-case relation between BFS and DFS is $N_{\text{BFS}} \approx N_{\text{DFS}} \times$:
 (a) b (b) $\frac{b+1}{b-1}$ (c) b^d (d) $\frac{1}{2}$
- Q29.** With $b = 5$, when DFS first reaches depth 5 the number of nodes on OPEN is:
 (a) 21 (b) 25 (c) 3125 (d) 5
- Q30.** Which property does BFS have that DFS lacks?
 (a) linear space (b) guaranteed shortest path (c) exponential time (d) uses a stack
- Q31.** Depth Bounded DFS is:
 (a) complete and optimal (b) linear in space but not complete (c) exponential in space (d) the same as BFS
- Q32.** In DFID (with the count), the search stops without a solution when:
 (a) the depth bound reaches 10 (b) two successive iterations return the same count (c) OPEN becomes a queue (d) CLOSED is empty
- Q33.** In the Siddharth Sagar example, DFID *with* CLOSED returns:
 (a) S-B-D-G (b) S-A-C-E-G (c) S-A-C-D-G (d) no path
- Q34.** For a full b -ary tree with I internal nodes, the number of leaves L is:
 (a) bI (b) $(b - 1)I + 1$ (c) b^I (d) $I + 1$
- Q35.** DFID's cost relative to BFS is approximately:
 (a) b (b) $\frac{b}{b-1}$ (c) b^2 (d) d
- Q36.** DFID was invented by:
 (a) chess programmers (b) logicians (c) roboticists (d) compiler writers
- Q37.** "The monster that AI fights" according to the slides is:
 (a) the Turing test (b) combinatorial explosion (c) the frame problem (d) overfitting

- Q38.** DFS, BFS and DFID are called blind because:
 (a) they ignore MoveGen (b) their exploration order does not depend on where the goal is
 (c) they cannot detect the goal (d) they use random choices
- Q39.** A computer 9 times faster lets a blind search with $b = 3$ go deeper by:
 (a) 9 levels (b) 3 levels (c) 2 levels (d) 1 level
- Q40.** Which data-structure change makes BFS's OPEN update $O(1)$ per node instead of $O(|\text{OPEN}|)$?
 (a) using a hash set for CLOSED (b) using a deque/queue instead of list append (c) using recursion (d) storing depths

Answers and explanations

Q	Ans	Explanation
1	b	The graph exists only through MoveGen; it is generated on demand.
2	b	Domain functions MoveGen and GoalTest (plus a state representation).
3	c	15 states on the ring plus (0, 5, 3) in the centre.
4	b	From (0, 5, 3) you can reach only (5, 0, 3) or (3, 5, 0).
5	a	The 5-litre jug holds 3 litres; $t = \min(3, 3 - 0) = 3$, so it empties into the 3-litre jug: (5, 0, 3). (Pouring the 8-litre jug into the 3-litre jug would give (2, 3, 3).)
6	b	Parity splits the $9!$ arrangements into two unreachable halves.
7	b	Moves are named by the direction the <i>tile</i> moves.
8	b	$16 - 6 = 10$.
9	c	Goat across, back, lion (or cabbage) across, goat back, cabbage (or lion) across, back, goat across.
10	c	N-queens seeks a final board; the others seek a path.
11	c	Fix the start, $(n - 1)!$ orders, each tour counted in both directions.
12	b	No record of visited nodes, so cycles repeat forever.
13	b	Lines 5–6: removed from OPEN and added to CLOSED when picked.
14	b	"It simply returns node G"; the path needs nodePairs.
15	b	Each node is added exactly once; shortest path is not implied.
16	b	(currentNode, parentNode).
17	c	$\text{tail}[3, 2, 1] = [2, 1]$, $\text{tail}[2, 1] = [1]$, $\text{head}[1] = 1$.
18	a	$[n] ++ [u, t] = [n, u, t]$.
19	c	$h = 101$, $t = [102, 103]$.
20	b	New pairs at the front: stack behaviour.
21	b	FIFO: oldest first.
22	b	Uses OccursIn on OPEN and CLOSED.
23	b	Each node is paired with the common parent.
24	b	Case 1: identical trees, different orders.
25	b	DFS cycles S, A, C, B, S...; BFS reaches depth 2 regardless.
26	b	One node per level, levels 0 to d .
27	b	All of levels $0..d - 1$, then one more.
28	b	$\frac{b^d(b+1)/(2(b-1))}{b^d/2}$.
29	a	$d(b - 1) + 1 = 5 \cdot 4 + 1 = 21$.
30	b	BFS inspects in order of depth.
31	b	The goal may lie beyond the bound.
32	b	The same count means no new nodes: the space is exhausted.

Q	Ans	Explanation
33	b	D gets closed via S–A–C–D, blocking S–B–D–G.
34	b	Edges counted as $L + I - 1$ and as bI .
35	b	$(L + I)/L \approx b/(b - 1)$.
36	a	Stated on the slide; it gives anytime move choice.
37	b	CombEx.
38	b	Same sweep/spiral wherever the goal is.
39	c	$\log_3 9 = 2$ extra levels.
40	b	Appending to a linked list walks it; a queue enqueues in $O(1)$.

Next week: heuristic search — using domain knowledge to decide which node to pick from OPEN, escaping local optima, and stochastic local search.